O'REILLY®

Second Edition

Early Release

RAW & UNEDITED

Fundamentals of Software Architecture

An Engineering Approach

Mark Richards & Neal Ford

Fundamentals of Software Architecture

by Mark Richards and Neal Ford

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are

also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional

sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Louise Corrigan

Development Editor: Sarah Grey

Production Editor: Christopher Faucher

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

January 2020: First Edition

March 2025: Second Edition

Revision History for the Early Release

2024-08-23: First Release

See http://oreilly.com/catalog/errata.csp?isbn=9781098175511 for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Fundamentals of Software Architecture*,

the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publishers' views.

While the publisher and the authors have used good faith efforts to ensure that the information and

instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility

for errors or omissions, including without limitation responsibility for damages resulting from the use of

or reliance on this work. Use of the information and instructions contained in this work is at your own

risk. If any code samples or other technology this work contains or describes is subject to open source

licenses or the intellectual property rights of others, it is your responsibility to ensure that your use

thereof complies with such licenses and/or rights.

978-1-098-17541-9

Brief Table of Contents *(Not Yet Final)*

PART I

Foundations

It understand important trade-offs in architecture, developers must understand

some basic concepts and terminology concerning components, modularity coupling,

and communication

Architectural Thinking

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at *grey@oreilly.com*.

Architectural thinking is about seeing things with an architect's eye—in other words, from an architectural point of view. Understanding how a particular change might impact overall scalability, paying attention to how different parts of a system interact, and knowing which third-party libraries and frameworks would be most appropriate for a given situation are all examples of thinking architecturally.

Being able to think like an architect first involves understanding what software architecture is and the differences between architecture and design. It then involves having a wide breadth of knowledge to see solutions and possibilities that others do not see; understanding the importance of business drivers and how they translate to architectural concerns; and understanding, analyzing, and reconciling trade-offs between various solutions and technologies.

In this chapter we explore these aspects of thinking like an architect.

Architecture Versus Design

Be a moment and picture your dream house in your mind. How many floors does it have? Is the roof flat or peaked? Is it a big sprawling single-story ranch house or a multistory contemporary home? How many bedrooms does it have? All of these things define the overall structure of the house—in other words, its architecture. Now take a moment to picture the inside of the house. Does it have carpeting or hardwood floors? What color are the walls? Are there floor lamps or lights hanging from the ceiling? All of these things relate to the design of the house.

Similarly,software architecture is less about a system's appearance and more about its structure, whereas design is more about a system's appearance and less about its structure. For example, the choice to use microservices defines the structure and shape of the system (its architecture), whereas the look and feel of the user interface (UI) screens defines the design of the system.

But what about decisions such as whether to break apart a service into smaller parts or checking into a UI framework? Unfortunately most decisions such as these fall somewhere on a spectrum between architecture and design, making it difficult to determine what should be considered architecture.

Leveraging the following criteria can help determine whether something is more about architecture or more about design:

- Is it more strategic in nature or more tactical?
- How much effort will it take to change or construct?
- How significant are the trade-offs?

These factors are shown in Figure 1-1, illustrating the spectrum between architecture and design to help determine where a decision lies and who should have the responsibility for it.

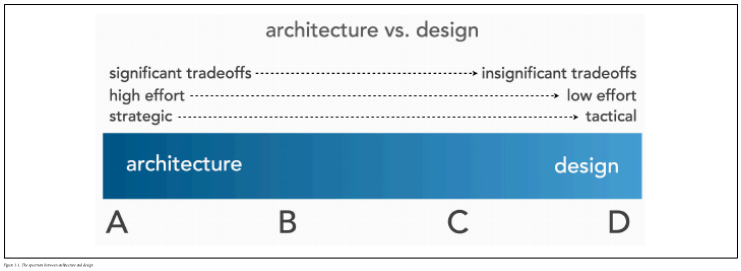


Figure 1-1. The spectrum between architecture and design

Strategic versus Tactical Decisions

The more strategic a decision, the more architectural it becomes. Conversely the more tactical a decision, the more it likely to be about design. Strategic decisions are generally long term, whereas tactical decisions are generally short term and usually independent of other actions or decisions.

One good way to determine whether a decision is more strategic or tactical is to consider the following question:

How much thought and planning is involved in the decision?

A decision that takes a couple of minutes is more likely to be tactical and hence more about design, whereas a decision requiring weeks of planning is likely to be

more strategic and hence more about architecture.

How many people are involved in the decision?

A decision made alone or with a colleague is likely more tactical and on the design side of the spectrum, whereas a decision requiring lots of meetings with many different stakeholders is likely more strategic and on the architectural side of the spectrum.

Is the decision a long-term vision or a short-term action?

A decision that is likely to change soon is usually tactical in nature and would be more about design, whereas one that will last a very long time is typically more strategic and more about architecture.

While these questions are a bit subjective, they nevertheless help in determining whether something is strategic or tactical, and thus more about architecture or design.

Level of Effort

In his famous paper "Who needs an architect", software architect Martin Fowler writes that architecture is the stuff that hard to change! The harder something is to change, generally the more effort is required, placing that decision or activity towards the architectural side of the spectrum. Conversely something that requires minimal effort to implement or change places it more on the design side of the spectrum.

For example, moving from a monolithic layered architecture to microservices would require significant effort, so it would be more about architecture. Rearranging fields on a screen would require minimal effort, and therefore be more about design.

Significance of Trade-offs

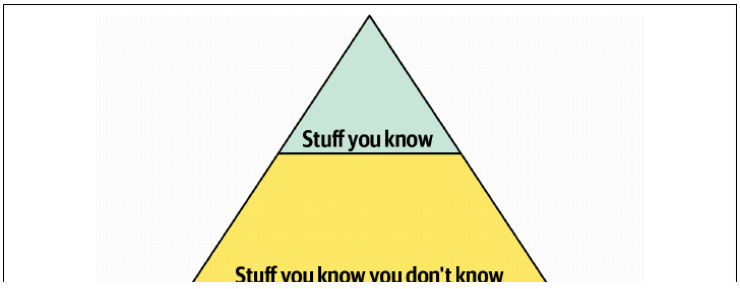
Analyzing the trade-offs of a particular decision can help a lot in determining whether it is more about architecture or design. The more significant the trade-offs are, the more architectural the decision tends to be. For example, choosing to use the microservices architecture style provides better scalability, agility, elasticity, and fault tolerance. However, this architecture is highly complex, is very expensive, has poor data consistency, and doesn't perform well due to service coupling. These are some pretty significant trade-offs. We can conclude that this decision is more on the architectural side of the spectrum than design.

Even design decisions have trade-offs. For example, breaking apart a class file provides better maintainability and reusability—at the cost of managing more classes. These trade-offs are not overly significant (especially compared to microservices) trade-offs), so this decision is more on the design side of the spectrum.

Technical Breadth

Unlike developers, who must have a significant amount of technical depth to perform their jobs, software architects must have a significant amount of technical breadth to see things from an architectural point of view. Technical depth is all about having deep knowledge of a particular programming language, platform, framework, protocol, etc., and so on, whereas technical breadth is all about knowing a little bit about a lot of things.

To better understand the difference, consider the knowledge pyramid shown in Figure 1-2. It encapsulates all the technical knowledge in the world, which can be broken down into three levels: *stuff you know*, *stuff you know you don't know*, and *stuff you don't know you don't know*.



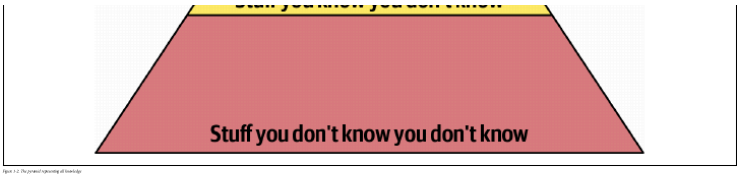


Figure 1-2. The pyramid representing all knowledge

Baby on Rails, that expertise vanishes if they ignore Ruby on Rails for a year or two.

Stuff you know includes the technologies, frameworks, languages, and tools technologists keeping things at the top of the pyramid requires a time investment to maintain gains one on a daily basis to perform their jobs (such as a Java programmer knowing expertise). This top part represents the individual's technical depth: the things they

level). People good, or even expert, at all these things. Notice that this level of knowl- know really well.

edge (represented by the top part of the pyramid) is the smallest and contains the

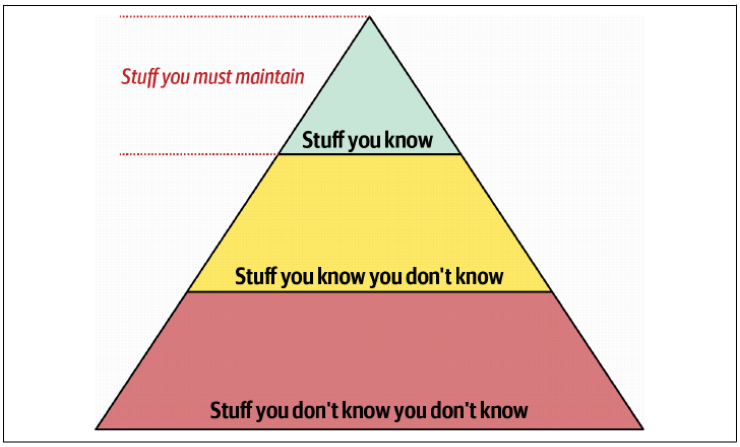
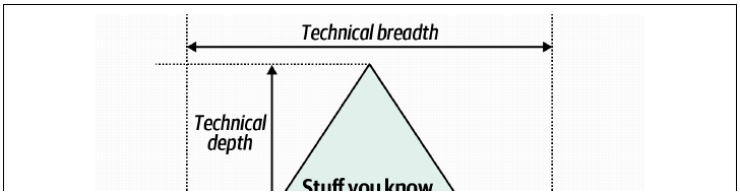


Figure 1-3. Developers must maintain expertise in certain a

However the nature of knowledge changes as developers transition into the architect role. A large part of the value of an architect is that they have a broad understanding of technology and how to use it to solve particular problems. For example, it's better for an architect to know that five solutions exist for a particular problem than to have singular expertise in only one. The most important parts of the pyramid for architects are the top and middle sections; how far the middle section penetrates into the bot- tom section represents an individual's technical breadth, as shown in Figure 1-4.



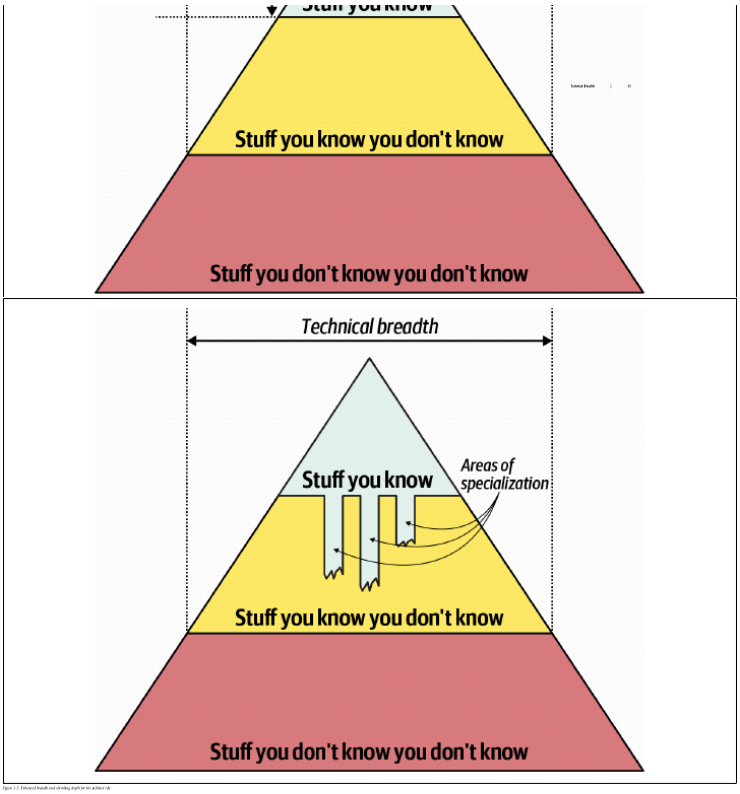


Figure 1-1. Enhanced breadth and shrinking depth for the architect role

Our knowledge pyramid illustrates how fundamentally different the roles of architect and developer are. Developers spend their whole careers honing expertise. Transitioning to the architect role means a shift to that perspective, which many individuals find difficult. This in turn leads to two common dysfunctions: first, an architect tries to maintain expertise in a wide variety of areas, succeeding to none of them and working themselves ragged in the process. Second, it manifests as take-expert-for—the mistaken sensation that your outdated information is still cutting edge. We see this often in large companies where the developers who founded the company have moved into leadership roles, yet still make technology decisions using ancient criteria (see "Frozen Caveman antipattern" on page 16).

Architects should focus on technical breadth so that they have a larger canvas from which to draw arrows. Developers transitioning to the architect role may have to change the way they view knowledge acquisition. Balancing their portfolio of knowledge regarding depth versus breadth is something every developer should consider throughout their career.

Frozen Caveman antipattern

A behavioral antipattern commonly observed in the wild, the Frozen Caveman antipattern, describes architects who revert to their pet technical concern for every architecture. For example, one of Neal's colleagues worked on a system that featured a

centralized architecture. Each time they delivered the design to the client architects, the persistent question was "But what if we lose Italy?" Several years before, a freak communication problem had prevented the client headquarters from communicating with its stores in Italy causing great inconvenience. While the chances of this recurring were extremely small, the architects had become obsessed with this particular architectural characteristic.

Generally this antipattern manifests in architects who have been burned in the past by a poor decision or unexpected occurrence, making them particularly cautious about anything related. While risk assessment is important, it should be realistic as well. Understanding the difference between genuine and perceived technical risk is part of the ongoing learning process. Thinking like an architect requires overcoming these Frozen Caveman ideas and experiences, seeing other solutions, and asking more relevant questions.

Analyzing Trade-Offs

Thinking like an architect is about seeing trade-offs in every solution, technical or otherwise, and analyzing those trade-offs to determine the best solution. The reason this is one of the critical activities of an architect (and hence part of architectural thinking) is exemplified through the following quote by Mark (one of your authors):

> Architecture is the stuff you can't Google or ask an AI engine about.
>
> —Mark Richards

Everything in architecture is a trade-off, which is why the famous answer to every architecture question in the universe is "It depends." While this answer might be annoying, it is unfortunately true. You cannot Google the answer or ask an AI engine or large language model (LLM) whether REST or messaging would be better for your system or whether microservices is the right architecture style for your new product, because the answer *does* depend. It depends on the deployment environment, business drivers, company culture, budgets, timeframes, developer skill set, and dozens of other factors. Everyone's environment, situation, and problem will be different. That's why architecture is so hard. To quote Neal, your other author:

> There are no right or wrong answers in architecture—only trade-offs.
>
> —Neal Ford

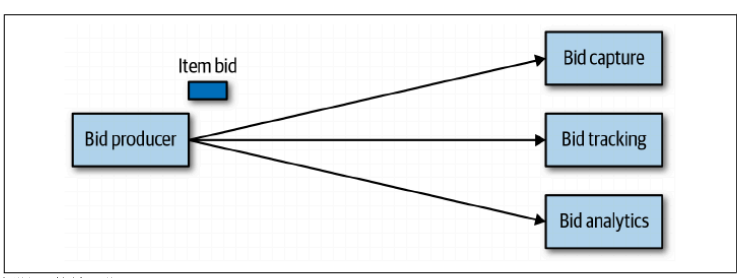


Figure 1-6. Auction system example of a trade-off—queues or topics?

For example, consider an item auction system (Figure 1-6) where online bidders bid on items up for auction. The Bid Producer service generates a bid from the bidder and then sends that bid amount to the Bid Capture, Bid Tracking, and Bid Analytic services. The architect could do this using queues in a point-to-point messaging fashion, or by using a topic in a publish-and-subscribe messaging fashion. Which one should they choose? They ask Google the answer. Architectural thinking requires them to analyze the trade-offs associated with each option and select the best service based

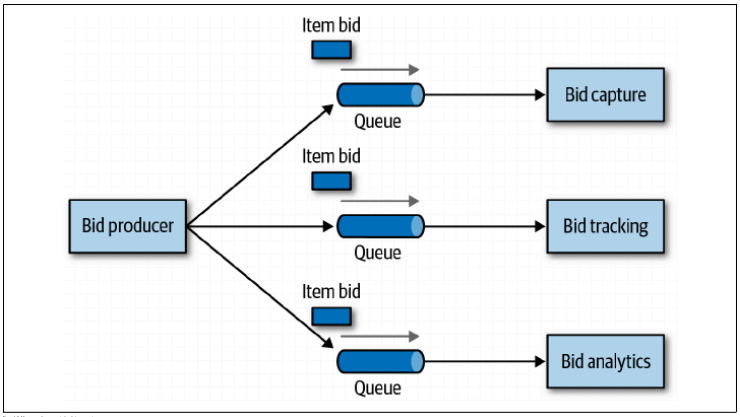


Figure 1-6. Using queues for communication between services

The clear advantage (and seemingly obvious solution) to this problem in Figure 1-7 is architectural extensibility. The Bid Producer service only requires a single connection to a topic. Compare that to the queue solution in Figure 1-6, where the Bid Producer needs to connect to three different queues. If a new service called Bid History were to be added to this system (to provide each bidder with a history of all of their bids), no changes would be needed to the existing services or infrastructure. The new Bid History service could simply subscribe to the topic that already contains the bid information.

However with the queue option shown in Figure 1-6, the Bid History service would require a new queue, and the Bid Producer would need to be modified to add an additional connection to the new queue. The point here is Bid using queues means that adding new bidding functionality requires significant changes to the services and infrastructure, whereas with the topic approach, no changes to the existing infrastructure are needed. Also, the Bid Producer is less coupled in the topic option, where the Bid Producer doesn't know how the bidding information will be used or by which services. In the queue option, the Bid Producer knows exactly how the bidding information will be used (and by whom), and hence is more coupled to the system.

So far this trade-off analysis seems to make it clear that the topic approach using the publish-and-subscribe messaging model is the obvious and best choice. However to quote Rich Hickey, the creator of the Clojure programming language:

> Programmers know the benefits of everything and the trade-offs of nothing. Architects need to understand both.
>
> —Rich Hickey

Thinking architecturally means not only looking at the benefits of a given solution, but also analyzing the associated negatives, or trade-offs. Continuing with the auction system example, a software architect would analyze the negatives of the topic solution as well as the positives. In Figure 1-7, you can see that with a topic, anyone can access

bidding data, which introduces a possible issue with data access and data security.

However in the queue model illustrated in Figure 3-8, the data sent to the queue can only be accessed by the specific consumer receiving that message. If a rogue service tries to listen to the queue, the bidding service would not receive those bids, and a notification would immediately be sent about the loss of data (and possible security breach). In other words, it is very easy to secure a topic, but not a queue.

In addition to the security issue, the topic solution in Figure 3-7 only supports homogeneous contracts. All services that receive the bidding data must accept the same data contract and set of bidding data. In the queue option, each consumer can have its own contract, specific to the data it needs. For example, suppose the new Bid History service requires the current asking price along with the bid, but no other service needs that information. In this case, the contract would need to be modified, impacting all other services using that data. In the queue model, this would be a separate channel, and thus a separate contract that does not impact any other service.

Another disadvantage of the topic model is that it does not support monitoring the number of messages in the topic, as it can support auto-scaling capabilities. However with the queue option, each queue can be monitored individually and programmatic load balancing applied to each bidding consumer so that each can be automatically scaled independently.Now that this trade-off is technology-specific the Advanced Message Queuing Protocol (AMQP) can support programmatic load balancing and monitoring because of the separation between an exchange (what the producer sends to) and a queue (what the consumer listens to).

Given this fuller trade-off analysis, which is the better option?

It depends! Table 3-1 summarizes those trade-offs.

Table 3-1. Trade-offs for topics

Topic advantages	Topic disadvantages
Architectural extensibility	Data access and data security concerns
Service decoupling	No heterogeneous contracts
	Monitoring and programmatic scalability
	Scalability

Again, *everything* in software architecture has trade-offs: advantages and disadvantages. Thinking like an architect means analyzing these trade-offs, then asking questions like, "Which is more important: extensibility or security?" An architect choice will always depend on the business drivers, environment, and a host of other factors.

Understanding Business Drivers

Thinking like an architect also means understanding the business drivers required for the success of the system and translating those requirements into architecture characteristics such as scalability performance, and availability This is a challenging task that requires the architect to have some business-domain knowledge and healthy collaborative relationships with key business stakeholders. We devoted four chapters in the book to this specific topic: in ???, we define various architecture characteristics. In ???, we describe ways to identify and qualify architecture characteristics. In ???, we describe how to measure each characteristic to ensure the business needs of the system are met. And finally in ??? we discuss the scope of architectural characteristics and how they relate to coupling.

Balancing Architecture and Hands-On Coding

One of the difficult tasks an architect faces is how to balance hands-on coding with software architecture. We firmly believe that every architect should code and should maintain a certain level of technical depth (see "Technical Breadth" on page 12). While this may seem like an easy task, it is sometimes rather difficult to accomplish.

Our first tip for anyone striving to balance hands-on coding with being a software architect is avoiding the *Bottleneck Trap*. The Bottleneck Trap antipattern occurs when an architect takes ownership of code within the critical path of a system (usually the underlying framework code or some of the more complicated parts) and becomes a bottleneck to the team. This happens because the architect is not a full-time developer and therefore must balance development work (like writing and testing source code) with the architect role (drawing diagrams, attending meetings, and well, attending more meetings).

One way to avoid the Bottleneck Trap is for the architect to delegate the critical parts of the system to others on the development team and then focus on coding a minor piece of business functionality (such as a service or UI screen) one to three iterations down the road. This has three positive effects. First, the architect gains hands-on experience by writing production code, while avoiding becoming a bottleneck on the team. Second, the critical path and framework code are distributed to the development team (where they belong), giving the team ownership and a better understanding of the harder parts of the system. Third, and perhaps most important, the architect is writing the same business-related source code as the development team.

This helps them better identify with the development team's pain points around processes, procedures, and the development environment (and hopefully work to improve those things).

Suppose, however that the architect can't develop code with the development team. How can they still remain hands-on and maintain some level of technical depth? The following are some tips and techniques for architects who want to continue deepening their technical abilities:

Frequent proofs of concept

Doing frequent proofs-of-concept (POCs) requires the architect to write source code; it also helps validate an architecture decision by taking the implementation

details into account. For example, suppose an architect gets stuck trying to choose between two caching solutions. One effective way to help make this decision is to develop a working example in each caching product and compare the results. This allows the architect to see firsthand the implementation details and the amount of effort required to develop the full solution. It also allows them to better compare architectural characteristics between the different caching solutions, such as scalability/performance, and overall fault tolerance.

Wherever possible, the architect should write the best production-quality code they can. We recommend this practice for two reasons. First, quite often, "throw-away" proof-of-concept code goes into the source code repository and becomes the reference architecture or guiding example for others to follow. The last thing any architect would want is for their sloppy throwaway code to be touted as representing their typical quality of work. Second, writing production-quality proof-of-concept code means getting practice at writing quality well-structured code, rather than continually developing bad coding practices by creating quick, sloppy POCs.

Addressing Technical Debt

Another way architects can remain hands-on is to tackle some technical debt, freeing the development team to work on the critical functional user stories. Technical debt is usually low priority so if the architect doesn't get the chance to complete a technical debt task within a given iteration, it will be the end of the world and generally won't impact the success of the iteration.

Proof of Technology

Similarly, working on bug fixes within an iteration is another way of maintaining hands-on coding skills while helping the development team. While certainly not glamorous, this technique allows the architect to identify issues and weaknesses within the code base and possibly the architecture.

Automate

Leveraging automation by creating simple command-line tools and analyzers to help the development team with their day-to-day tasks is another great way to maintain hands-on coding skills while making the development team more effective. Look for repetitive tasks the development team performs and automate them. They'll be grateful for the automation. Some examples are unit-related source validators to help check for specific coding standards not found in other first-turn, automated checklists, and repetitive manual-code-refactoring tasks.

Automation in the form of architectural analysis and fitness functions to ensure the vitality and compliance of the architecture is another great way to stay "hands-on" for example, an architect can write Java code in ArchUnit (in the Java platform to automate architectural compliance, or custom fitness functions to ensure architectural compliance while gaining hands-on experience. We talk about these techniques in Ch7.

Do code reviews

A final technique to remain hands-on as an architect is to do frequent code reviews. While the architect is not actually writing code, this at least keeps them involved in the source code. Further benefits of code reviews include ensuring compliance with the architecture and identifying mentoring and coaching opportunities on the team.

There's More to Architectural Thinking

This chapter forms the foundational aspects of beginning to think like an architect. However there's much more to thinking like an architect than what is described in this chapter. Thinking like an architect involves understanding the overall structure of a system (we cover that topic next in Chapter 2), understanding business needs, concerns, and translating those concerns to architectural characteristics (we have four chapters on that topic), and finally seeing a system through its logical components—the building blocks of a system (we discuss that topic in Ch3).

CHAPTER 2

Modularity

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter please reach out to the editor at qwang@oreilly.com.

Architects and developers have struggled with the concept of modularity for quite some time, as is evident from the following quote:

> 90% of the words I written about software architecture] are spent extolling the benefits of "modularity" and little if anything, is said about how to achieve it.
>
> —Glenford J Myers, *Composite/Structured Design*, (Van Nostrand Reinhold

Different platforms offer different reuse mechanisms for code, but all support some way of grouping related code together into modules. While this concept is universal in software architecture, it has proven slippery to define. A casual internet search yields dozens of definitions, with no consistency (and some contradictions). As mentioned earlier this is a new problem. However, because no recognized definition exists, we must jump into the fray and provide our own definitions for the sake of consistency throughout the book.

Understanding modularity and its many incarnations in the development platform of choice is critical for architects. Many of the tools we have to analyze architectures (such as metrics, fitness functions, and visualizations) rely on modularity and related concepts. Modularity is an organizing principle. If an architect designs a system without paying attention to how the pieces were together, that system will present myriad difficulties. If we use a physics analogy, software systems model complex systems, which tend toward entropy (or disorder). In a physical system, energy must be added to preserve order. The same is true for software systems: architects must constantly expend energy to ensure structural soundness, which won't happen by accident.

Preserving good modularity exemplifies our definition of an implicit architecture characteristic: virtually no project requirements explicitly ask the architect to ensure good modular distinction and communication, but sustainable code bases do require the order and consistency that this brings.

Modularity Versus Granularity

Developers and architects often use the terms modularity and granularity interchangeably yet their meanings are very different. Modularity is about breaking systems apart into smaller pieces, such as moving from a monolithic architecture style (like the traditional n-tiered layered architecture) to a highly distributed architecture style like microservices. Granularity on the other hand, is about the size of those pieces—how big a particular part of the system (or service) should be. However as stated in the following quote by one of your authors, it's with granularity where architects and developers get into trouble:

> Embrace modularity but beware of granularity
>
> —Ruth Malan

Granularity causes services or components to be coupled to one another creating complex, hard-to-maintain architecture antipatterns like *Inspector Gadget Architecture*, *Distributed Monolith*, and the famous *Big Ball of Distributed Hell*. The trick to avoiding those architectural antipatterns is to pay attention to granularity and the overall level of coupling between services and components.

Defining Modularity

Merriam-Webster defines a module as *each* of a set of standardized parts or independent units that can be used to construct a more complex structure. By contrast, in this book, we use *modularity* to describe a logical grouping of related code, which could be a group of classes in an object-oriented language or a group of functions in a structured or functional language. Developers typically use modules as a way to group related code together. For example, the *com.mycompany.customer* package in Java should contain things related to customers. Most languages provide mechanisms for modularity (package in Java, namespace in .NET, kind of—ish).

Modern programming languages feature a wide variety of packaging mechanisms and many developers find it difficult to choose between them. For example, in many modern languages, developers can define behavior to functions/methods, classes, or packages/namespaces, each with different visibility and scoping rules. Some languages complicate this further by adding programming constructs, such as the *method object protocol*, to provide even more extension mechanisms.

Architects must be aware of how developers package things, because packaging has important implications in architecture, for example, if several packages are tightly

coupled, reusing one of them for related work becomes more difficult.

important.) For example, lumping a large number of classes together in a monolithic application may be convenient; however, when it comes time to restructure the architecture, the coupling encouraged by loose partitioning can impede efforts to break the monolith apart. That's why it's useful to talk about modularity as a concept, separate from the physical separation that a particular platform forces or implies.

It is worth discussing the general concept of a namespace, which is separate from the technical implementation in the .NET platform that is also called a namespace. Developers often need precise, fully qualified names for software assets (components, classes, and so on) to separate them from each other. The most obvious example that people use every day is the internet, which relies on unique, global identifiers tied to IP addresses.

Most languages have some modularity mechanism that doubles as a namespace to organize things like variables, functions, or methods. Sometimes the module structure is reflected physically: Java package structures, for example, must reflect the directory structure of the physical class files.

Modular reuse before classes

Developers who trained in the days before object-oriented languages may puzzle over why there are so many different separation schemes. Much of the reason has to do with backward compatibility—not of code, but of how developers think about things. In March 1968, the journal *Communications of the Association for Computing Machinery (ACM)* published a paper from computer scientist Edsger Dijkstra entitled "Go To Statement Considered Harmful." He denigrated the common use of the `GOTO` statement, common in programming languages at the time, because it allowed nonlinear leaping around within code, making reasoning and debugging difficult.

Dijkstra's paper helped usher in the era of *structured* programming languages, exemplified by Pascal and C, which encouraged a deeper change in thinking about how things fit together. Developers quickly realized that most programming languages offered no good way to group like things together logically. Thus, the short era of *modular* languages was born, such as Modula (Pascal creator Niklaus Wirth's next language) and Ada. These languages embraced the programming construct of the *module*, much as we think about packages or namespaces today (but without the classes).

However, the modular programming era was short-lived because object-oriented languages became popular and offered new ways to encapsulate and reuse code. Still, language designers realized the utility of modules and retained them in the form of packages and namespaces. Many languages still contain compatibility features that seem odd today but were introduced to support these different paradigms. For example, Java supports modular paradigms (via packages and package-level initialization

A language with no name conflicts: Java 1.0

The original designers of Java had extensive experience dealing with name conflicts and clashes, which were common in programming platforms at the time. Java 1.0 used a clever hack to avoid ambiguity when two classes had the same name—such as if the problem domain included a catalog *order* and an installation *order*, both named *order* but with very different connotations (and classes). The Java designers' solution was to create the `package` namespace mechanism, along with a requirement that the physical directory structure must match the package name. Because filesystems won't allow two files with the same name to reside in the same directory, this leveraged the operating system's inherent features to ambiguity and name conflicts. Thus, the original `classpath` in Java contained only directories.

However, as the language designers discovered, forcing every project to have a fully formed directory structure was cumbersome, especially as projects became larger. Plus, building reusable assets was difficult: frameworks and libraries had to be "exploded" into the directory structure. In the second major release of Java (1.2, but called Java 2), designers added the `jar` mechanism, which allows an archive file to act as a directory structure on a classpath. For the next decade, Java developers struggled with getting the `classpath` exactly right, as a combination of directories and JAR files. Their original intent had been broken: now two JAR files *could* create conflicting names on a classpath. This is why Java developers of that era tend to have numerous war stories about debugging class loaders.

Measuring modularity

Since modularity is so important, architects need tools to help them better understand it. Fortunately researchers have created a variety of language-agnostic metrics for this purpose. We'll focus here on three key concepts: *cohesion*, *coupling*, and *connascence*.

Cohesion

Cohesion refers to the extent to which a module's parts should be contained within the same module. In other words, it measures how related the parts are to one another. An ideal cohesive module is one where all parts are packaged together; breaking them into smaller pieces would require coupling the parts together via calls between modules to achieve useful results. The cautionary tale of modularity as it relates to cohesion are exemplified in the following quote by Larry Constantine:

> Attempting to divide a cohesive module would only result in increased coupling and decreased readability
>
> —Larry Constantine, Structured Design (2008)

Computer scientists have defined a range of cohesion, measured from best to worst:

Functional cohesion

Every part of the module is related to the other, and the module contains everything essential it needs to function.

Sequential cohesion

Two modules interact: one outputs data that becomes the input for the other.

Communicational cohesion

Two modules form a communication chain in which each operates on information and/or contributes to some output. For example, one adds a record to the database and the other generates an email based on that information.

Procedural cohesion

Two modules must execute code in a particular order.

Temporal cohesion

Modules are related based on timing dependencies. For example, many systems have a list of seemingly unrelated things that must be initialized at system startup; these different tasks are *temporally cohesive*.

Logical cohesion

The data within modules is related logically but not functionally. For example, consider a module that converts information from text, serialized objects, or streams into some other format. Its operations are related, but the functions are

quite different. A common example of this type of cohesion exists in virtually every Java project in the form of the `StringUtils` package, a group of static methods that operate on `String` but are otherwise unrelated.

Coincidental cohesion

The elements in a module are unrelated other than being in the same source file. This represents the most negative form of cohesion.

Despite its many variants, *cohesion* is a less precise metric than *coupling*. Often, a particular module's degree of cohesion is determined at the discretion of a particular architect. Consider this module definition:

```
Customer Maintenance
```

`Customer Maintenance`

- `add customer`
- `update customer`
- `get customer`
- `notify customer`
- `get customer orders`
- `cancel customer orders`

Should the last two entries reside in this module? Or should the developer create two separate modules? Here's what that would look like:

`Customer Maintenance`

- `add customer`
- `update customer`
- `get customer`
- `notify customer`

`Order Maintenance`

- `get customer orders`
- `cancel customer orders`

Which is the correct structure? As always, it depends:

- Are these the only two operations for `Order Maintenance`? If so, it may make sense to collapse those operations back into `Customer Maintenance`.
- Is `Customer Maintenance` expected to grow much larger? If so, perhaps developers should look for opportunities to extract behavior.
- Does `Order Maintenance` require so much knowledge of `Customer` information that separating the two modules would require a high degree of coupling to make it functional? (This relates back to the prior Larry Constantine quote.)

These questions represent the kind of trade-off analysis that lies at the heart of a software architect's job.

Computer scientists have developed a good structural metric to determine cohesion—] specifically Lack of Cohesion (which is a bit surprising, given how subjective this characteristic is). A well-known set of metrics named the Chidamber and Kemerer Object-Oriented Metrics Suite measures particular aspects of object-oriented software systems. It includes many common code metrics, such as Cyclomatic Complexity (see ???) and several important coupling metrics, discussed in "Coupling" on page 32.

Chidamber and Kemerer have also developed a Lack of Cohesion in Methods (LCOM) metric, which measures the structural cohesion of a module. The initial version appears in Equation 2-1.

Equation 2-1. LCOM, version 1

$$LCOM = \begin{cases} |P| - |Q|, & \text{if } |P| > |Q| \\ 0, & \text{otherwise} \end{cases}$$

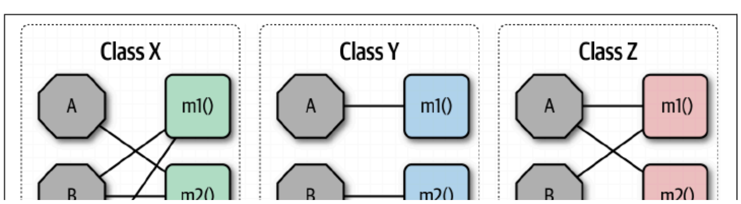
In this equation, P increases by 1 for any method that doesn't access a particular shared field; Q decreases by 1 for methods that *do* share a particular shared field. If you find this formulation confusing, we sympathize—and it gradually become even more elaborate. The second variation, introduced in 1996 (thus the name *LCOM96B*), appears in Equation 2-2.

Equation 2-2. LCOM96B

$$LCOM96b = \frac{1}{a} \sum_{j=1}^{a} \frac{m - \mu(A_j)}{m}$$

We won't bother untangling the variables and operators in Equation 2-2 because the following written explanation is clearer. Basically the LCOM metric exposes incidental coupling within classes. A better definition of LCOM would be "the sum of sets of methods not shared via sharing fields."

Consider a class with private fields `a` and `b`. Many of the methods only access `a`, and many other methods only access `b`. The *sum* of the sets of methods not shared via sharing fields (`a` and `b`) is high; therefore, this class incurs a high LCOM score, indicating a significant *lack of cohesion in methods*.



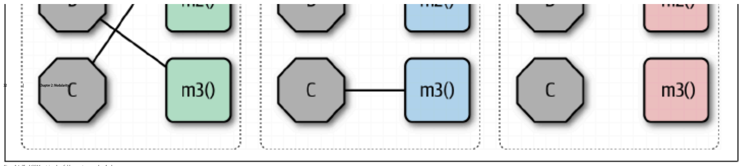


Figure 2-1. The LCOM metric, where fields are octagons and methods are squares

Consider the three classes shown in Figure 2-1. Here, fields appear as single letters inside octagons and methods appear as blocks. In Class X, the LCOM score is low indicating good structural cohesion. Class *Y*, however, lacks cohesion; each of the field/method pairs in Class Y could appear in its own class without affecting the system behavior. Class Z shows mixed cohesion; the last field/method combination could be refactored into its own class.

tools for virtually every platform that allow architects to analyze the coupling characteristics of code.

> **Why such similar names for coupling metrics?**
>
> Why are two critical metrics in the architecture world that represent *opposite concepts* named virtually the same thing, differing in only the vowels that sound the most alike? These terms originate from the book *Structured Design*. Borrowing concepts from mathematics, Yourdon and Constantine coined the now-common terms *afferent* and *efferent* coupling. They should really have been called *incoming* and *outgoing* coupling, but the authors leaned toward mathematical symmetry rather than clarity. Developers have come up with several mnemonics to help out. For instance, *a* appears before *e* in the English alphabet, just as *incoming* appears before *outgoing*. The letter *e* in *efferent* matches the initial letter in *exit*, which helps in remembering that it represents outgoing connections.

Metrics: Abstractness, Instability, and Normalized Distance from the Main Sequence

While component coupling has raw value to architects, several other derived metrics allow for deeper evaluation. The metrics discussed in this section were created by software engineer Robert C. Martin, and apply widely to most object-oriented languages.

Abstractness is the ratio of abstract artifacts (abstract classes, interfaces, and so on) to concrete artifacts (implementations). The Abstractness metric measures a code base's degree of abstractness versus implementation. For example, on one end of the scale would be a code base with no abstractions, just a huge, single function of code (as in a single `main()` method). The other end of the scale would be a code base with too many abstractions, making it difficult for developers to understand how things wire together (For example, it takes developers a while to figure out what to do with an `AbstractSingletonProxyFactoryBean` abstract class, given its many layers of abstraction and ambiguous name.)

The formula for abstractness appears in Equation 2-3.

Equation 2-3. Abstractness

$$A = \frac{\sum m^a}{\sum m^c + \sum m^a}$$

Architects calculate Abstractness by calculating the ratio of the sum of abstract artifacts to the sum of the concrete and abstract ones. In the equation, m^a represents

abstract elements (interfaces or abstract classes) with the module, and m represents concrete elements (non-abstract classes). This metric looks for the same criteria. The easiest way to visualize this metric is to consider an application with 5,000 lines of code, all in one `main()` method. Its Abstractness numerator would be 1, while the denominator would be 5,000, yielding an Abstractness score of almost 0. That how this metric measures the ratio of abstractions in code.

Another derived metric, Instability, is defined as the ratio of efferent coupling to the sum of both efferent and afferent coupling, as shown in Equation 2-4.

Equation 2-4. Instability

$$I = \frac{C_e}{C_a + C_e}$$

In the equation, c_e represents efferent (or outgoing) coupling, and c_a represents afferent (or incoming) coupling.

The Instability metric determines the volatility of a code base. A code base that exhibits its high degrees of instability breaks more easily when changed, because of high coupling. For example, if a class calls in many other classes to delegate work, the calling class will show high susceptibility to breakage if one or more of the called methods changes.

Distance from the Main Sequence

One of the first holistic metrics architects have for architectural structure is *Distance from the Main Sequence*, a derived metric based on Instability and Abstractness, shown in Equation 2-5.

Equation 2-5. Distance from the main sequence

$$D = |A + I - 1|$$

In the equation, A = Abstractness and I = Instability.

Note that both Abstractness and Instability are fractions whose results will always fall between 0 and 1 (except in some extreme cases). Thus, graphing the relationship produces the graph in Figure 2-2.

Figure 2-2. The main sequence defines the ideal relationship between Abstractness and Instability

The *Distance* metric imagines an ideal relationship between abstractness and instabil-

ity ; classes that fall near this idealized line exhibit a healthy mixture of these two com-
lower left-hand corner, as illustrated in Figure 2-4, enters the *Zone of Pain*: code with

peting concerns. For example, graphing a particular class allows developers to
too much implementation and not enough abstraction becomes brittle and hard to

calculate the Distance from the Main Sequence metric, illustrated in Figure 2-3.
maintain.

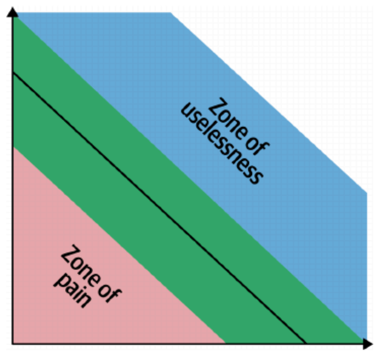


Figure 2-4. The zones of u selessness and pain

Many platforms provide tools to calculate these measures, which assist architects when analyzing code bases to get familiar with them, prepare for a migration, or assess technical debt.

The limitations of metrics

While the industr y has a few code-level metrics that provide valuable insight, our tools are extremely blunt compared to analysis tools in other engineering disciplines. Even metrics derived directly from the structure of code require interpretation. For example, Cyclomatic Complexity_ (see ???) measures complexity in code bases, but this metric cannot distinguish between *essential* complexity (the code is complex because the underlying problem is complex) and *accidental complexity* (the code is more complex than it should be). Virtually all code-level metrics require interpretation, but it is still useful to establish baselines for critical metrics such as Cyclomatic Complexity so that architects can assess which type the codebase exhibits. We discuss setting up just such tests in ???.

Yourdon and Constantine's *Structured Design*, published in 1979, predates the popularity of object-oriented languages. It focuses instead on structured programming constructs, such as functions (not methods). It also defines other types of coupling that are outdated because of modern program language design. Object-oriented pro-

gramming introduced additional concepts that overlay afferent and efferent coupling, including a more refined vocabular y to describe coupling called *connascence*.

Connascence

Meilir Page-Jones's book *What Every Programmer Should Know about Object-Oriented Design* (Dorset House, 1996) created a more precise *language* to describe different types of coupling in object-oriented languages. Connascence isn a coupling metric

like afferent and efferent coupling—rather it represents a language that helps architects describe different types of coupling more precisely (and understand some common consequences of types of coupling).

Two components are *connascent* if a change in one would require the other to be modified in order to maintain the overall correctness of the system. He distinguished two types of connascence: *static* and *dynamic*.

Static connascence

Static connascence refers to source-code-level coupling (as opposed to execution-time coupling, covered in "Dynamic connascence" on page 38). Architects view static connascence as the *degree* to which something is coupled via either afferent or efferent coupling. There are several types of static connascence:

Connascence of Name
: Multiple components must agree on the name of an entity.

Method names and method parameters are the most common way that code bases are coupled and the most desirable, especially in light of modern refactoring tools that make system-wide name changes trivial to implement. For example, developers no longer change the name of a method in an active code base but rather *refactor* the method name using modern tools, affecting the change throughout the code base.

Connascence of Position
: Multiple components must agree on the order of values.

This is an issue with parameter values for method and function calls, even in languages that feature static typing. For example, if a developer creates a method `void updateSeat(String name, String seatLocation)` and calls it with the values `updateSeat("14D", "Ford, N")`, the semantics aren't correct, even if the types are.

Connascence of Algorithm
: Multiple components must agree on a particular algorithm.

A common case for *Connascence of Algorithm* occurs when a developer defines a security hashing algorithm that must run and produce identical results on both the server and client to authenticate the user. Obviously this represents a high degree of coupling—if any details of either algorithm change, the handshake will no longer work.

Connascence of Type
: Multiple components must agree on the type of an entity.

This type of connascence refers to the common tendency in many statically typed languages to limit variables and parameters to specific types. However this capability isn't purely for statically typed languages—some dynamically typed languages also offer selective typing, notably Clojure and Clojure Spec.

Connascence of Meaning
: Multiple components must agree on the meaning of particular values. It is also called *Connascence of Convention*.

The most common obvious case for this type of connascence in code bases is hard-coded numbers rather than constants. For example, it is common in some languages to consider defining somewhere that `int TRUE = 1; int FALSE = 0`. Imagine the problems that would arise if someone flipped those values.

Dynamic connascence

The other type of connascence Page-Jones defines is *dynamic connascence*, which analyzes calls at runtime. The types of dynamic connascence include:

Connascence of Execution
: The order of execution of multiple components is important.

Consider this code:

```
email = new Email();
email.setRecipient("foo@example.com");
email.setSender("me@me.com");
email.send();
email.setSubject("whoops");
```

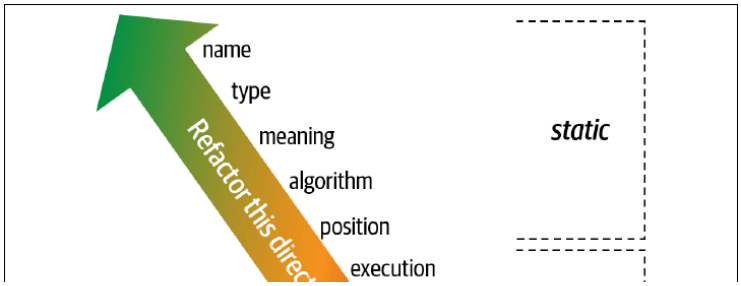
It won't work correctly because certain properties must be set in a specific order.

Connascence of Timing
: The timing of the execution of multiple components is important.

The common case for this type of connascence is a race condition caused by two threads executing at the same time, affecting the outcome of the joint operation.

Connascence of Values
: Several values depend on one another and must change together.

Consider a case where a developer has defined a rectangle by defining four points to represent its corners. To maintain the integrity of the data structure, the devel-



one cannot randomly change one of the points without considering the impact on the other points to preserve the shape of the rectangle.

A more common and problematic case involves transactions, especially in distributed systems. In a system designed with separate databases, when someone needs to update a single value across all of the databases, the values must either all change together or not change at all.

Connascence of Identity

Multiple components must reference the same entity.

A common example of *Connascence of Identity* involves two independent components that must share and update a common data structure, such as a distributed queue.

Connascence Properties

Connascence is an analysis framework for architects and developers, and some of its properties help ensure that we use it wisely.Three connascence properties include:

Strength

Architects determine the *strength* of a system connascence by the ease with which a developer can refactor its coupling. Some types of connascence are demonstrably more desirable than others, as shown in Figure 2-5. Refactoring toward better types of connascence can improve the coupling characteristics of a code base.

Architects should prefer static connascence to dynamic because developers can determine it by simple source-code analysis, and because modern tools make it trivial to improve static connascence. For example, *Connascence of Meaning* could be improved by refactoring to *Connascence of Name*,creating a named constant rather than a magic value.

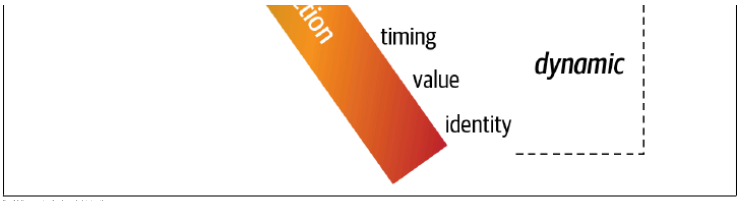


Figure 2-3. Connascence strength can be a good refactoring aide

Locality

> The locality of a system's connascence measures how proximal (close) its modules
>
> are to each other in the code base. *Proximal* code (code in the same module) typi-
>
> cally has easier and higher forms of connascence than more separated code. Se-
>
> parate modules or code bases). In other words, forms of connascence that
>
> would indicate poor coupling when the components are far apart are fine when
>
> the components are closer together. For example, if two classes in the same mod-
>
> ule have Connascence of Meaning, it is less damaging to the code base than if
>
> those classes were in different modules.

Architects were largely unaware of the importance of this observation when the author first published it. In modern terms, he is suggesting that architects should limit the scope of implementation details. *High coupling* as narrowly as practical, which is the same advice derived from Domain-driven design's bounded context idea.

The architectural observation is the same— limit implementation coupling. Martin Page described a good design principle which was misunderstood more fully via DDD.

> It is a good idea to consider strength and locality together. Stronger forms of connas-
>
> cence within the same module *represent less leakage overall* than the same connascence
>
> spread apart.

Degree

> The *degree* of connascence relates to the size of the impact if changing a class in a particular module—does that change impact a few classes or many? Lesser degrees of connascence require fewer changes to other classes and modules and hence damage code bases less. In other words, having high dynamic connascence isn't terrible if an architect only has a few modules. However, code bases tend to grow, making a small problem correspondingly bigger in terms of change.

In *What Every Programmer Should Know about Object-Oriented Design* (Dorset House, 1996), Page-Jones offers three guidelines for using connascence to improve system modularity:

- Minimize overall connascence by breaking the system into encapsulated elements
- Minimize any remaining connascence that crosses encapsulation boundaries
- Maximize connascence within encapsulation boundaries

The legendary software architecture innovator Jim Weirich, who repopularized the concept of connascence, offers two great rules from his talk on *Connascence Examined* during the *Emerging Technologies for the Enterprise* conference in 2012:

> Rule of Degree: Convert strong forms of connascence into weaker forms of connascence.
>
> Rule of Locality: As the distance between software elements increases, use weaker forms of connascence.

Architects benefit from learning about connascence for the same reason it's beneficial to learn about design patterns—connascence provides more precise language to describe different types of coupling. For example, an architect can tell someone "we need a service and there can only be one instance," or they can tell someone, "we need a Singleton service"--the Singleton design pattern nicely encapsulates the context and solution to a common problem with a simple name.

Similarly, when performing a code review, an architect can instruct a developer, "Don't add a magic string constant in the middle of a method declaration. Extract it as a constant instead." Or they could say, "You have *Connascence of Meaning*; refactor it to *Connascence of Name*."

From Modules to Components

We use the term *module* throughout this book as a generic name for bundles of related code. However, most architects refer to modules as *components*, the key building blocks for software architecture. This concept of a *component*, and the corresponding analysis of logical or physical separation, has existed since the earliest days

of computer science, yet developers and architects still struggle to achieve good outcomes.

We'll discuss deriving components from problem domains in ???, but we must first discuss another fundamental aspect of software architecture: architectural characteristics and their scope.

The difference between an architecture style and an architecture pattern can be confusing. We define an architecture style as the overarching structure of how the user interface and backend source code are organized (such as within layers of a monolithic deployment or separately deployed services) and how that source code interacts with a database. Architecture patterns, on the other hand, are lower-level design structures that help form specific solutions within an architecture style (such as how to achieve high scalability or high performance within a set of operations or between sets of services).

Understanding architecture styles occupies much of the time and effort for new architects because they share importance and abundance. Architects must understand the various styles and the trade-offs encapsulated within each to make effective decisions; each architecture style embodies a well-known set of trade-offs that help an architect make the right choice for a particular business problem.

CHAPTER 3

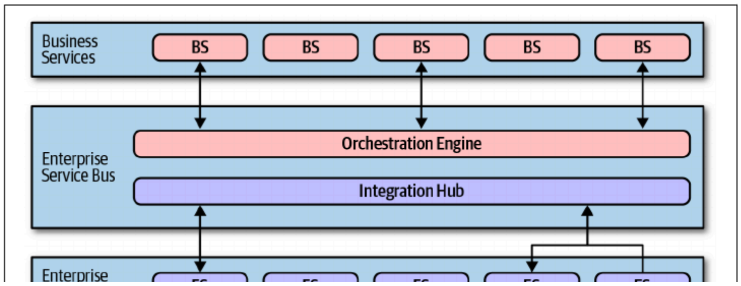
Orchestration-Driven Service-Oriented Architecture

> **A Note for Early Release Readers**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 17th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at *sgrey@oreilly.com*.

Architecture styles make sense to architects in the context of the era when they evolved, but lose relevance in later eras, much like art movements. *Orchestration-driven service-oriented architecture* (SOA) exemplifies this tendency. The external forces that often influence architecture decisions, combined with a logical but ultimately disastrous organizational philosophy, doomed this architecture to irrelevance. However, it provides a great example of how a particular organizational idea can make logical sense yet hinder the most important parts of the development process. It illustrates one of the dangers of ignoring our First Law: *everything in software architecture is a trade-off*.

Topology

The topology of orchestration-driven SOA is shown in Figure 3-1.



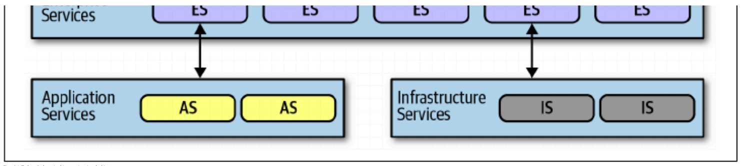


Figure 3-1. Topology of orchestration-driven service-oriented architecture

these services? If so, then the service is at the appropriate level of granularity. How-
Not all examples of this style of architecture have the exact layers illustrated in
ever, while a developer might need a method like `CreateCustomer` to carry out a busi-
Figure 3-1, but they all follow the same idea of establishing a taxonomy of services
ness process like `ExecuteTrade`, `CreateCustomer` is at the wrong level of abstraction
within the architecture, each layer with a specific, well-defined responsibility.
for a business service. The company isn't in the *business* of creating customers, but it
Service-oriented architecture is a distributed architecture. The exact demarcation of
needs to create customers in order to execute trades.
boundaries isn't shown in Figure 3-1 because it varies based on organization and
These service definitions contain no code—just input, output, and sometimes schema
tools: some of the parts of the taxonomy may exist inside an application server.
information. Business users and/or analysts define these service signatures, hence the
Orchestration-driven SOA centers on a specific taxonomy of services, with different
name *business services*.
technical responsibilities within the architecture and roles dedicated to specific layers.

Enterprise Services
Taxonomy

The *enterprise services* contain fine-grained shared implementations. Typically a team
The driving philosophy behind this architecture is a specific type of abstraction and
of developers builds atomic behavior around particular business domains, such as
enterprise-level reuse. Many large companies were annoyed at how much they had to
`CreateCustomer` or `CalculateQuote`, and transactional entities, such as `Customer`,
continually rewrite software. They arrived at a strategy that seemed to solve that
`Order`, and `LineItem`. These enterprise services are the building blocks that make up
problem gradually by creating a strict *service taxonomy*, with well-defined layers and
the business services, tied together via the orchestration engine.
corresponding responsibilities. Each layer of the taxonomy supports the dual goals of
It is worth noting the abstraction differences between business and enterprise serv-
ultimate abstraction and reuse.
ices. While business services are quite coarse-grained, enterprise services are fine-
grained and meant to capture different types of abstractions, workflows, and entities.

Business services

The architect's goal in creating enterprise services is to create perfectly encapsulated
Business services sit at the top of this SOA and provide the entry point for business
building blocks of isolated business functionality that can be freely composed into
processes. For example, services like `ExecuteTrade` or `PlaceOrder` represent the cor-
more complex business workflows.
rect scope of behavior for these services. One litmus test common at the time was:
can an architect answer "yes" to the question "Are we in the business of..." for each of
While that's a laudable goal, architects find that the ideal sweet spot of abstraction
between all these forces is elusive at best and likely impossible because of numerous
competing trade-offs. Ultimately, like other technically partitioned architectures, this

architecture attempts a strict separation of responsibility from which flows from the
imperative of reuse. The idea is that if developers can build fine-grained enterprise
services at just the correct level of granularity, the business won't have to rewrite that
part of the business workflow again. Gradually the business will build up a collection
of reusable assets in the form of reusable enterprise services—in theory, anyway.

Unfortunately, the dynamic nature of reality and the evolutionary impact of the soft-
ware development ecosystem defy these attempts. Business components aren't like
construction materials, where solutions last decades. Markets, technology changes,
engineering practices, and a host of other factors confound attempts to impose stabil-
ity on the software world.

Application services

Not all services in the architecture require the same level of granularity or reuse as the
enterprise services. *Application services* are one-off, single-implementation services.
For example, perhaps one application needs geolocation, but the organization doesn't

want to take the time or effort to make that a reusable service. An application service,
typically owned by a single application team, solves that problem.

Infrastructure services

Infrastructure services supply operational concerns, such as monitoring, logging,
authentication, authorization, and so on. These services tend to be concrete imple-
mentations, owned by a shared infrastructure team that works closely with opera-

tions. Architects philosophy in running this architecture revolves around technical

partitioning, so it makes sense that they would build separate infrastructure services.

Orchestration engine and message bus

The orchestration engine forms the heart of this distributed architecture, stitching together the business service implementations using orchestration, including features like transactional coordination and message transformation. The orchestration engine defines the relationship between the business and enterprise services, how they map together and where transaction boundaries lie. It also acts as an integration hub allowing architects to integrate custom code with packages and legacy software systems. This combination of features highlights the modern-day use of tools like Message flow (or ESBs). While most architects consider it a bad idea to build an entire architecture around ESBs, they are immensely useful in integration-heavy environments. Where architects must combine an integration hub and orchestration worlds. Thus, message flow goes through the engine even for internal calls, as shown in Figure 3-2.

engine, why not use a tool that already includes them? This points to another topic—

tant skill to develop as an architect—how to discern the true uses of tools, separate

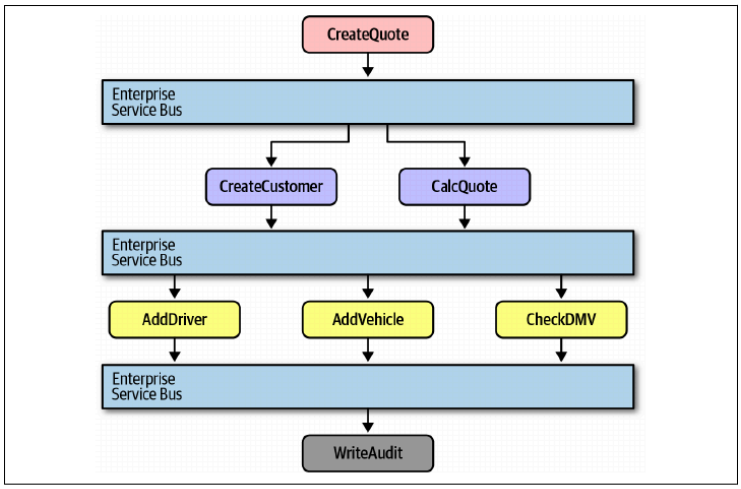


Figure 3-2. Message flow with service-oriented architecture

In Figure 3-2, the `CreateQuote` business-level service calls the service bus, which defines the workflow. The workflow consists of calls to `CreateCustomer` and `CalcQuote` requests, each of which also makes calls to application services. The service bus acts as the intermediary for all calls within this architecture, serving as both an integration hub and orchestration engine.

Reuse . . . and coupling

One of the primary goals of the architects who first utilized this architecture was reuse at the service level—the ability to gradually build business behavior that they could incrementally reuse over time. They were instructed to find reuse opportunities as aggressively as possible.

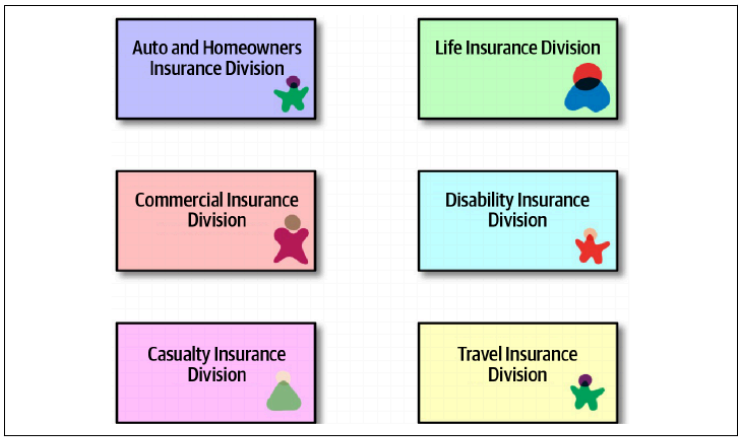
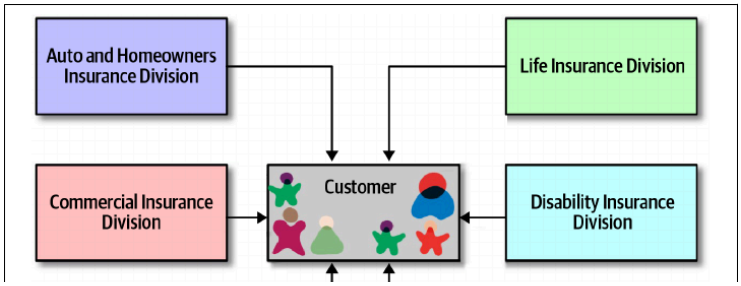


Figure 3-3. Finding reuse opportunities in a service-oriented architecture

For example, consider the situation illustrated in Figure 3-3. An architect realizes that each of six divisions within an insurance company contains a notion of Customer. Therefore, the proper SOA strategy entails extracting the Customer parts into a reusable service and then allowing the original services to reference the canonical Customer service. This is shown in Figure 3-4, here the architect has isolated all customer behavior into a single Customer service, achieving the obvious reuse goals.



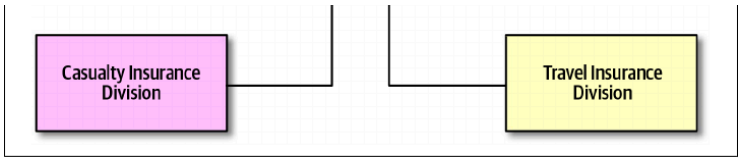


Figure 3-4. Building conceptual representations in service-oriented architectures

Architects only slowly realized the negative trade-offs of this design. First, when a team builds a system primarily around reuse, it also incurs a huge amount of coupling between components. After all, reuse is implemented via coupling. For example, in Figure 3-4, a change to the `Customer` service ripples out to all the other services. This makes even incremental change risky: each change has a potentially huge ripple effect. That in turn requires coordinated deployments, holistic testing and other drags on engineering efficiency.

Another negative side-effect of consolidating behavior into a single place: consider the case of auto and disability insurance in Figure 3-4. It support a single `Customer` service, each division must include all the details the organization knows about its customers. Auto insurance requires a driver license, which is a property of the person, not the vehicle. Therefore, the `Customer` service will have to include details about drivers' licenses that the disability insurance division cares nothing about. If the disability insurance users must deal with the extra complexity of a single customer definition. In many ways, Domain Driven Design instances cry *analog holistic reuse* drivers from experiences with these kinds of architectures.

Perhaps the most damaging evolution about orchestration-driven SOA is the impracticality of building an architecture on focused on technical partitioning. While this makes sense from a separation and reuse philosophy standpoint, it is a practical nightmare.

For example, developers commonly work on tasks like "add a new address line to `CatalogCheckout`". Domain concepts like `CatalogCheckout` are spread so thinly

throughout this architecture that they were virtually ground to dust. In an SOA, this task could involve dozens of services in several different tiers, plus changes to a single database schema. What means, if the current enterprise services aren't defined at the correct transactional granularity, the developers will either have to change their design or build a nearly identical new service to change the transactional behavior be much for reuse.

Style specifics

Orchestration-driven SOA is mostly of historical interest to current-day architects; the lessons learned there building these architectures are an important part of the field evolution. However architects still find parts of it relevant in some integration architecture scenarios.

Service-oriented architecture appeared in the late 1990s, just as small companies of all kinds were growing into enterprises at a breakneck pace, merging with smaller companies, and requiring more sophisticated IT to accommodate this growth. However computing resources were scarce, precious, and commercial. Distributed computing had just become possible and necessary and many companies needed its scalability and other beneficial characteristics.

Many external drivers forced architects in this era toward distributed architectures with significant system constraints. Before open source operating systems were thought reliable enough for serious work, operating systems were expensive and licensed per machine. Similarly commercial database servers came with Byzantine licensing schemes, which sometimes caused application-server vendors (which offered database connection pooling) to react by battling with database vendors. Given that so many resources were expensive at scale, architects adopted a common philosophy of reusing as much as possible.

Those technical concerns meshed with organizational concerns about duplication of both infrastructures and workflows—because of frequent mergers and growth, organizations struggled with the variety and inconsistencies between core business entities. This made the goals of SOA attractive. Consequentlyarchitects embraced reuse in all forms as the dominant philosophy in this architecture, the side effects of which we cover in "Reuse at… and coupling" on page 28. This style of architecture also exemplifies how for architects can push the idea of technical partitioning, which can have good motivations but leads to bad consequences when taken to extremes.

Why so many *service* names?

Architects are often confused by the multitude of things in software architecture called services. We cover three different but vaguely styles even in this book: SOA in this chapter microservices in ??, and service-based architectures in ??. Part of the prob-

ben is the inflexibility of language to describe architectures. The other part lies in the constant evolution of the software development ecosystem. Service is a nice generic name for something that provides a, well, service, so architects tend to name the name. As styles evolve, we change what we name by service. For example, an entity service in an orchestration-driven SOA is different in virtually every way from a service in a microservices architecture (which is distinct from a service-based architecture). As annoying as it is, architects must often parse the context when the word service appears in a name, the term itself has suffered from semantic diffusion.

Data topologies for orchestration-driven SOA

Unlike many of the architecture styles we discuss in this book, the data topologies for orchestration-driven SOA aren't very interesting, given this style's historical origins. Even though it is a distributed architecture consisting of many parts, it generally uses a single (or a few) relational databases, which was the common practice in all distributed architectures in the late 1990s. Data transactionality was generally relegated to this architecture and away from databases: the message bus often included declarative transactional instructions for each of the entities within the topology, allowing developers, architects, or others to determine transactional behavior independently of the database or even the situational reuse of the entity.

Architects in this era considered data a foreign country. While it is an inevitable part of the plumbing in both SOA and event-driven architectures, back then, they treated it more as an integration point than as part of the problem domain.

Really? *Declarative Transactions?!?*

In reality, One of the "features" of many application servers in the heyday of orchestration-driven SOA was allowing configuration managers to change the transactional scope of individual entities, depending on the transactional context within which they wanted to operate. This was declared, of course, in XML, given the era and its love for verbose but easy-to-parse configuration formats. A portion of the declaration of an entity (called `EntityBeans`, a specialized type of database determinism transactional scope as it participates in workflows, which architects themselves declare to be transactional or not. In cases, the application server interacts with the database to create and manage database transactions that match the desired behavior of the entities and/or workflows.

This has largely failed, for two reasons. First, if a developer doesn't know what the transactional behavior will be at runtime, it adds considerable complexity to entities and dependencies. This forces developers to create almost identical versions of the entities, differing only in their transactional scope. Second, no matter how much sophistication vendors build into their message buses, edge cases constantly appear

where the myriad failure modes prevent the system from cleanly managing transactions, creating tangled messes of inconsistency for humans to untangle. Some complex, multifaceted features of systems (like transactions) cannot be cleanly abstracted away. Be many leaks in the abstraction prevent it from achieving reliability.

Cloud considerations for orchestration-driven SOA

Orchestration-driven SOA predates the cloud by several decades, so no considerations exists for building this architecture (in its original incarnation) in the cloud.

However, the current-day use of this style makes it a good integration architecture for cloud and on-premises services that must integrate and participate in workflows. As primarily an integration architecture, it works well with cloud-based services and facilities.

Governing orchestration-driven SOA

When this architecture was popular, modern-day holistic testing was uncommon. Even rarely tested SOA outside of formal quality-assurance-level testing, so tool and framework creators gave little consideration to facilitating testing the individual parts. Some testing frameworks emerged to create mocks and stubs for the massive machinery of the message bus and its related moving parts, but they were always cumbersome and inconsistent.

Governance suffered from the same limitations. The idea of automating architectural governance was even more foreign than automating testing. In this era, *governance* meant heavyweight frameworks, meetings, and code reviews—all manual.

However, architects still see ESBs strategically within organizations that need the particular mix of capabilities they offer. In particular, many companies have legacy systems that must interact with many modern systems, often combining results and aggregating behavior—all of which describes the central functionality of an ESB. In these scenarios, fitness functions can serve a critical role in preventing data or boundary contexts from "leaking" across parts of the ecosystem where they shouldn't appear.

For example, consider a system that uses an ESB to coordinate between an enterprise resource planning (ERP) package, an online sales tool, and more modern microservices-based accounting services. In this scenario, the system should only read from the ERP and sales systems and should write to the accounting microservices. Architects can first build a fitness function that ensures that all communication is written consistently in logs, then write something like the following fitness function (given here in pseudocode):

```
READ logs for ERP into ERP-logs for past 24 hours

READ logs for Sales into Sales-Logs for past 24 hours

FOREACH entry IN ERP-logs

IF 'operation' is 'update' and 'target' != 'accounting' THEN

raise Fitness function violation

"Invalid communication between integration points"

END IF

FOREACH entry IN Sales-logs

IF 'operation' is 'update' and 'target' != 'accounting' THEN

raise Fitness function violation

"Invalid communication between integration points"

END IF
```

This fitness function reads the log entries from both integration points to ensure that no update operations take place whose target isn't the document-ing system.

Using such fitness functions, architects can use tools like ESBs strategically while building guardrails around the places where teams typically misuse them.

Common risks for orchestration-driven SOA

At the end of the last century and the beginning of this one, the big risks for this architecture were primarily about how much it cost, how long it took to implement, and (a shocking surprise) how difficult these systems are to maintain and update. Many of these projects were very expensive multi-year endeavors, with critical decisions made high in the concept hierarchy. Rather than call these projects "failures", companies mostly just transformed them into integration architectures with better boundaries, more closely aligned with the ideas of DDD.

When architects use an ESB in a modern system as an integration facility the biggest risk is the slippery slope of allowing the ESB to gradually encapsulate the entire architecture. This is called the *Accidental SOA* antipattern: where an architect gradually and unintentionally builds a fully orchestration-driven SOA without realizing it. To avoid Accidental SOA, an architect must ensure reasonable encapsulation boundaries for orchestration and pay close attention to issues like transactional boundaries.

Team topologies considerations

Just so architects did not consider data topologies for orchestration-driven SOA, the name is true for team topologies, which was an unknown topic when this architecture style was popular.

In fact, the strict taxonomy of this style serves as a communication antipattern that *led* architects to develop the principles of team topologies. The goal in this architecture is extreme separation of responsibilities, with a corresponding separation of team members. Among the companies that adopted it, it was rare indeed for someone building business services to chat with someone building enterprise services. They

were expected to communicate through technical artifacts, such as contracts and interfaces. The level of abstraction in this style creates many integration layers, each implemented by different teams, using enterprise-level ticketing tools to communicate. It should be easy to see why developers find it time-consuming to build features in this style.

Style characteristics

Many of the criteria we now use to evaluate architecture styles were not priorities when orchestration-driven SOA was popular. The Agile software movement had just started and had not yet penetrated the large organizations likely to use this architecture.

A one-star rating in the characteristics ratings table in Figure 3-3 means the specific architecture characteristic sink will supported in the architecture, whereas a five-star rating means the architecture characteristic is one of the style strongest features. Definitions for the characteristics identified in the scorecard can be found in Ch1.

SOA is perhaps the most technically partitioned general-purpose architecture ever attempted! In fact, the backlash against its disadvantages of this structure led to more modern architectures, such as microservices. SOA has a single quantum, even though it is a distributed architecture, for two reasons. First, it generally uses a single data base or just a few creating coupling points across many different consumers. Second, and more importantly, the orchestration engine acts as a giant coupling point—no part of the architecture can have different characteristics than the mediator that orchestrates all behavior. Thus, this architecture manages to find the disadvantages of both monolithic and distributed architectures.

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★

Fault tolerance	★★★★
Modularity	★★★★★
Overall cost ART: replace star with dollar sign, move the 3rd position in the table	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★

Figure 3-5. Ratings for our size-oriented architecture

Modern engineering goals such as deployability and testability score disastrously in this architecture, both because they are poorly supported and because those were not important (or even aspirational) goals when it was developed.

This architecture does support some goals, such as elasticity and scalability despite the difficulties in implementing them. Tool vendors poured enormous effort into making these systems scalable by building session replication across application servers and other techniques. However this being a distributed architecture, performance has never been a highlight, because each business request is split across so much of the architecture.

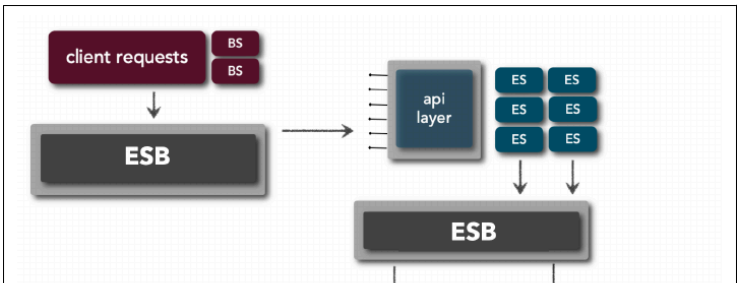
Because of all these factors, simplicity and cost have the inverse of the relationship most architects would prefer.Orchestration-driven SOA was an important milestone because it taught architects the practical limits of technical partitioning and how difficult distributed transactions can be in the real world.

Examples and use cases

The primary examples of this architecture existed in the late 1990s and early 2000s in many large enterprises. They have gradually been displaced by more agile and domain-based distributed architectures, like microservices. But large enterprises have realized that change is inevitable and that software isn't static and needs to change with market forces and new capabilities.

Architects built orchestration-driven SOA architectures to try to achieve effective reuse across large organizations, but eventually realized how difficult their strict and elaborate taxonomy makes implementing common changes and updates. For example, a common domain change might be to update details about a single entity. On a lucky day developers might only need to change components in the Enterprise Service layer to do the job. However on a bad day (one where enterprise architects realize business stakeholders didn't anticipate this type of change), developers might have to update four or five layers in the architecture, making highly coupled changes. In such, architects working in this style dread hearing the word change because it requires deep analysis and the scope of the work is not variable.

As we mentioned in "Governing orchestration-driven SOA on page 56", architects still use the building blocks of orchestration-driven SOA (such as the ESB), particularly for integration architectures. For example, an ESB (includes both an integration hub (to facilitate communication, protocol, and contract transformation) and an orchestration engine (to allow architect to build workflows between various integration endpoints). Because the orchestration-driven SOA includes many layers of indirection, it allows architects to implement enterprise services as integration points, as package software, or as bespoke code, among other possibilities, as illustrated in Figure 5-4.



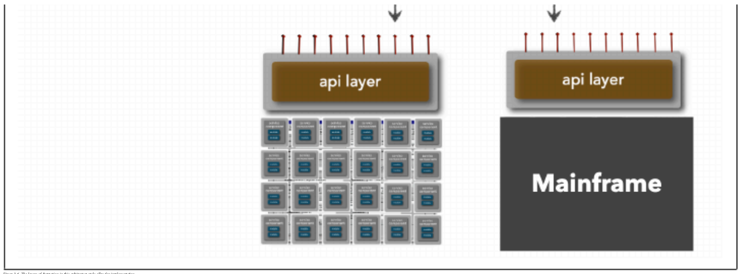


Figure 3-6. The layers of abstraction in this architecture style allow for implementation flexibility.

In Figure 3-6, client requests use the message bus to determine which enterprise services to call, in what order to call them, and what information to aggregate. The enterprise services, in turn, communicate via APIs to custom code, old systems, or package software, and so on.

Orchestration-driven SOA represents an interesting innovation for how architects handle problems of integration at scale within the constraints of their ecosystem. For example, since most organizations weren't using open source operating systems when this architecture was popular, alternative architectures like microservices were impossibly expensive. Architects should learn from past approaches. We can continue using the parts that still make sense, while internalizing the lessons of what failed and why

An effective software architect must not only understand the technical aspects of software architecture, but also the primary techniques and soft skills necessary to think like an architect, guide development teams, and effectively communicate the architecture to various stakeholders. This section of the book addresses the key techniques and soft skills necessary to become an effective software architect.

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 21st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter please reach out to the editor at gward@pearson.

One of the core expectations of an architect is to make architectural decisions. Architectural decisions usually involve the structure of the application or system, but they may involve technology decisions as well, particularly when those technology decisions impact architectural characteristics. Whatever the context, a good architectural decision is one that helps guide development teams in making the right technical choices. Making architectural decisions involves gathering enough relevant information, justifying the decision, documenting the decision, and effectively communicating that decision to the right stakeholders.

Architectural Decision Antipatterns

The programmer Andrew Koenig defines an antipattern as something that seems like a good idea when you begin, but leads you into trouble. Another definition of an antipattern is a repeatable process that produces negative results. The three most common architectural decision antipatterns that are (and usually do) emerge when an architect makes decisions are the Covering Your Assets antipattern, the Groundhog Day antipattern, and the Email-Driven Architecture antipattern. Those three antipatterns usually follow a progressive flow: overcoming the Covering Your Assets antipattern leads to the Groundhog Day antipattern, and overcoming that leads to the Email-Driven Architecture antipattern. Making effective and accurate architectural decisions requires overcoming all three.

The Covering Your Assets antipattern

The Covering Your Assets antipattern occurs when an architect avoids or delays making an architectural decision out of fear of making the wrong choice. There are two ways to overcome this. The first is to wait until the last *responsible moment* to make

an important architectural decision: that is, when there's enough information to justify and validate the decision, but not so long that it holds up development teams or sends the architect into the *Analysis Paralysis* antipattern, where they are stuck forever in analyzing the decision. A good way to determine the last responsible moment is to ask when the cost of deferring the decision exceeds the risk associated with deciding. As illustrated in Figure 4-1, notice that in the early stages of the decision-making time scale, the cost (denoted by the solid line) is low because there's less time spent on making the decision, but the risk (denoted by the dotted line) is high because less is known about the problem or solution. Spending more time deferring the decision increases the cost, but also reduces risk because the architect can make a more complete analysis of the problem and possible alternatives. The time to make a decision is where these two factors intersect and the cost increase exceeds the reduced risk.

Another way to avoid this antipattern is to collaborate with development teams to ensure that the decision can be implemented as expected. This is vitally important because no architect can possibly know every single detail about any issue associated with a particular technology. By closely collaborating with development teams, the architect can respond quickly, gain more insight, and reduce the risk of the decision being the incorrect one.

To illustrate this point: Suppose you, as the architect, decide that all product-related reference data (such as product description, weight, and dimensions) should be cached in all service instances needing that information. You decide that this will be done using a read-only replicated cache, with the primary cache owned by the `Catalog` service. (A *replicated cache* (or *in-memory cache*) means that if there are any changes to product information or new products added, the `Catalog` service updates its cache, which is then replicated to all other services requiring that data through a replicated cache product.) Your justification for this decision is to reduce coupling between the services and to share data effectively without having to make an interservice call. However, the development teams implementing this architectural decision find that, due to some services' scalability requirements, this decision would require more in-process memory than is available. Because you are closely collaborating with these teams, you quickly become aware of the issue and adjust your architectural decision accordingly.

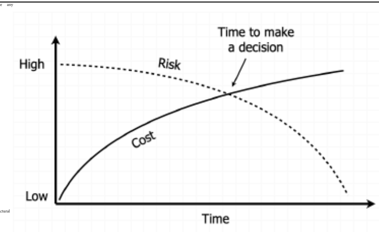


Figure 4-1. Last Responsible Moment

Groundhog Day antipattern

The Groundhog Day antipattern occurs when people don't know why an architect made a particular decision, so they keep discussing it over and over and over, never coming to a final resolution or agreement. It gets its name from the 1993 movie *Groundhog Day*, in which Bill Murray's character must relive February 2 over and over, every day.

This antipattern occurs because architects fail to justify their decision (or fail to jus-

tify it completely). It's important to provide both technical and business justifications for an architectural decision.

For example, say you decide to break apart a monolithic application into separate services. Your justification is to decouple the functional aspects of the application, so that each part of the application uses fewer virtual machine resources and can be maintained and deployed separately. While this is a good technical justification, what's missing is the *business* justification—in other words, why should the business pay for this architectural refactoring? A good business justification for this decision might be to deliver new business functionality faster, thereby improving time to market. Another might be to reduce the costs associated with developing and releasing new features.

Providing the business value is vitally important when justifying an architectural decision. It is also a good litmus test for determining whether the architectural decision should be made in the first place. If it doesn't provide any business value, perhaps the architect should reconsider the decision.

Four of the most common business justifications are cost, time to market, user satisfaction, and strategic positioning. Consider what's important to the business stakeholders. Justifying a particular decision based on cost savings alone might not be the right call if the business stakeholders are more concerned about time to market.

Email-Driven Architecture antipattern

Once an architect makes and fully justifies their decisions, another architecture anti-

pattern often emerges: Email-Driven Architecture . This antipattern occurs when

people lose or forget an architectural decision, or developers know that it been made

and therefore cannot possibly implement it. Overcoming this antipattern is all about

communicating architectural decisions effectively. Email is a great tool for communi-

cation, but it makes a poor document repository system.

Fortunately, an architect can easily avoid the Email-Driven Architecture antipattern

by learning how to communicate architectural decisions effectively. First, be sure not

to include the decision in the body of an email. Doing so creates multiple systems of

record for that decision, because each email contains a copy of the decision rather

than having it in only one place. Many such emails leave out important details about

the decision (including the justification), creating the Groundhog Day antipattern all

over again. Also, if that architectural decision is ever changed or superseded, fulfill-

ment to know whether all the relevant people receive the revised decision.

A better approach is to mention only the nature and context of the decision in the

body of the email and provide a link to the single system of record, where the archi-

tectural decision and corresponding details are stored (whether that a link to a wiki

page or a reference to a document in a filesystem).

Consider the following email about an architectural decision:

"Hi Sandra, We made an important decision regarding communication between

services that directly impacts you. Please see the decision using the following link..."

Notice, in the phrasing of the beginning of the first sentence, that the context is main-

tained (communication between services), but not the actual decision itself. The sec-

ond part of the first sentence is also important. If an architectural decision doesn't

directly impact the person, then why bother that person with it? This is a great litmus

test for determining which stakeholders (including developers) should be notified

directly of an architectural decision. The second sentence in this example provides a

link to the single location of the architectural decision, providing a single system of

record for decisions.

Architectural Significance

Many architects believe that if a decision involves a specific technology then it's not an architectural decision, but rather a technical decision. This is not always true. If an architect decides to use a particular technology because it directly supports a particular architecture characteristic (such as performance or scalability), then it's still an architectural decision.

Michael Nygard, a well-known software architect and author of *Release It! (2nd edition, Pragmatic Bookshelf, 2018)*, addresses the problem of what decisions an architect should be responsible for (and hence what constitutes an architectural decision) by coining the term *architecturally significant*. According to Michael, architecturally significant decisions are those that affect a system's structure, non-functional characteristics, dependencies, interfaces, or construction techniques.

Structure, in this context, refers to decisions that affect the architecture patterns or styles being used. For example, a decision by an architect to share code between a set of microservices impacts the bounded context of the microservice, and thus affects the structure of the system.

The *symbiosis* described characteristics are the architectural characteristics that are important for the system being developed or maintained. For example, if a choice of technology affects performance, and performance is an important aspect of the application, that choice becomes an architectural decision, even though it may specify a particular product, framework, or technology.

Dependencies refers to the coupling points between components and/or services within the system. Dependencies can affect architecture characteristics such as scalability, modularity, agility, testability, reliability and so on, so decisions about dependencies become architectural decisions.

Interfaces refer to how services and components are accessed and orchestrated: usually through a gateway integration hub, service bus, adapter or API, proxy. Making decisions about interfaces usually involves defining contracts, including versioning and deprecation strategies. Interfaces impact others using the system, and hence are architecturally significant.

Finally, *construction techniques* refers to decisions about platforms, frameworks, tools, and even processes that, although technical in nature, might impact some aspect of the architecture.

Architectural Decision Records

One of the most effective ways of documenting architectural decisions is through Architectural Decision Records (ADRs). Michael Nygard first evangelized ADRs in a 2011 blog post, and in 2017, ThoughtWorks' Technology Radar recommended the technique for widespread adoption.

An ADR consists of a short text file (usually one to two pages long) describing a specific architectural decision. While ADRs can be written using plain text or in a wiki page template, they are usually written in some sort of text document format, like AsciiDoc or Markdown.

Balsig is also available for managing ADRs. Mr. Pryce, coauthor of *Growing Object-Oriented Software, Guided by Tests* (Addison-Wesley 2009), has written an open source tool called ADR Tools that provides a command-line interface to manage ADRs, including numbering schemes, licenses, and superseded logic. Media Buys, a

software engineer from Germany provides some great examples of using ADR tools

to manage architectural decision records.

Basic Structure

The basic structure of an ADR consists of five main sections: *Title*, *Status*, *Context*,

Decision, and *Consequences*. We usually add two additional sections as part of the

basic structure: *Compliance* and *Notes*. The Compliance section is a space to think

about and document how the architectural decision will be governed and enforced

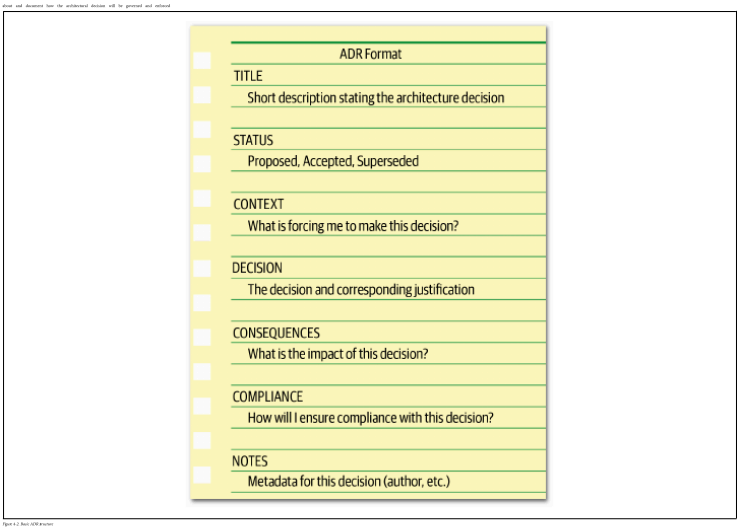


Figure 4-2. Basic ADR structure

Title

ADR titles are usually numbered sequentially and contain a short phrase describing

the architectural decision. For example, an ADR title describing a decision to use

asynchronous messaging between the Order Service and the Payment Service might

read: "42. Use of Asynchronous Messaging Between Order and Payment Services".

The title should be short and concise, but descriptive enough to remove any ambiguity

in about the nature and context of the decision.

Status

Every ADR has one of three statuses: *Proposed*, *Accepted*, or *Superseded*. *Proposed*

status means the decision must be approved by a higher-level decision maker or some

sort of architectural governance body (such as an architecture review board).

Accepted means the decision has been approved and is ready for implementation.

Superseded means the decision has been changed and superseded by another ADR.

Superseded status always assumes that the prior ADR status was *Accepted*; in other

words, a *Proposed* ADR would never be superseded by another ADR; it would be

modified until *Accepted*.

The *Superseded* status is a powerful way of keeping historical records of what deci-

sions have been made, why they were made at that time, what the new decision is, and

why it was changed. Ideally when an ADR has been superseded, it is marked with

the number of the decision that superseded it. Similarly the decision that supersedes

another ADR is marked with the number of the ADR it superseded.

For example, let say ADR 42 ("*Use of Asynchronous Messaging Between Order and*

Payment Services") is in *Approved* status. Due to later changes in the implementation

and location of the Payment Service, you decide that REST should now be used

between the two services. So therefore create a new ADR (Another 88) to document

this changed decision. The corresponding statuses look as follows:

ADR 42. Use of Asynchronous Messaging Between Order and Payment Services

Status: Superseded by 68

ADR 68. Use of REST Between Order and Payment Services

Status: Accepted, supersedes 42

The link and history trail between ADRs 42 and 68 lets you avoid the inevitable "what about using messaging?" question regarding ADR 68.

Another significant aspect of the Status section of an ADR is that it forces the architect and their boss or lead architect to discuss the criteria for approving an architectural decision and determine whether the architect can do so on their own, or whether it must be approved through a higher-level architect, architecture review board, or some other governing body.

Three good starting places for these conversations are cost, cross-team impact, and security. Cost should include software purchase or licensing fees, additional hardware costs, and the overall level of effort to implement the architectural decision. To estimate this, multiply the estimated number of hours to implement the architectural decision by the company's standard *full-time equivalency* (FTE) rate. The project owner or project manager usually has the FTE amount. This conversation might lead everyone to agree that, for example, the cost of the architectural decision exceeds a certain amount, then it must be set to Proposed status and approved by someone else. If the architectural decision impacts other teams or systems or has any sort of security implications, then it must be approved by a higher-level governing body or lead architect.

Once the team establishes and agrees on the criteria and corresponding limits (such as "costs exceeding $5,000 must be approved by the architecture review board"), document it well so that all architects creating ADRs will know when they can and cannot approve their own architectural decisions.

ADRs and Request for Comments (RFC)

Sending a draft ADR out for comments can help an architect validate their assumptions and assertions with a larger audience of stakeholders. An effective way of engaging developers and initiating collaboration is creating a new kind of status called *Request for Comments* (RFC) and specifying a deadline for reviewers to complete their feedback. Once that date is reached, the architect can analyze the comments, make any necessary adjustments to the decision, make the final decision, and set the status to Proposed (or Accepted, if the architect has authority to approve the decision).

An ADR with RFC status would look as follows:

STATUS

Request for Comments, Deadline 09 JAN 2026

Context

The Context section of an ADR specifies the forces at play. In other words, "What situation is forcing me to make this decision?" This section of the ADR allows the architect to describe the specific circumstances and concisely elaborate on the possible alternatives. If the architect is required to document the analysis of each alternative in detail, add an Alternatives section rather than including that analysis in the Context section.

The Context section also provides a place to document a specific area of the architecture itself. By describing the context, the architect is also describing the architecture. Using the example from the prior section, the Context section might read as follows: "The order service must pass information to the payment service to pay for an order currently being placed. This could be done using REST or asynchronous messaging."

Notice that this concise statement specifies not only the scenario but also the alternatives considered.

Decision

The Decision section of the ADR contains a description of the architectural decision, along with a full justification. Nygard recommends stating architectural decisions in an affirmative, commanding voice rather than a passive one. For example, the decision to use asynchronous messaging between services would read, "*We will use* asynchronous messaging between services." This is much better than "*I think* asynchronous messaging between services would be the best choice," which does not make clear what the decision is or even if a decision has even been made—only the opinion of the architect.

One of the most powerful aspects of the Decision section of ADRs is that it lets the architect emphasize the justification for the decision. Understanding *why* a decision was made is far more important than understanding *how* something works. This helps developers and other stakeholders better understand the rationale behind a decision, and therefore makes them more likely to agree with it.

To illustrate this point, let say you decide to use Google Remote Procedure Call (gRPC) to communicate between two particular services to reduce network latency due to very high responsiveness needs. Several years later, a new architect on the team decides to use REST instead of gRPC to make communication between services more consistent. Because the new architect doesn't understand why gRPC was chosen in the first place, their decision ends up having a significant impact on latency-causing slowness in upstream systems. If the new architect had access to an ADR, they would have understood that the original decision to use gRPC was meant to reduce latency (at the cost of tightly coupled services) and could have prevented this problem.

Consequences

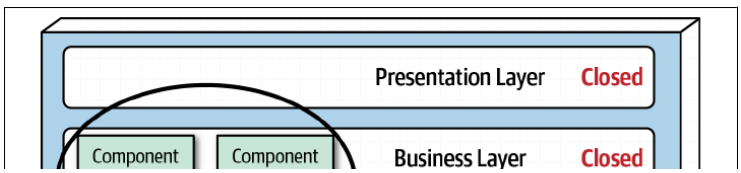
Every decision an architect makes has some sort of impact, good or bad. The Consequences section of an ADR forces an architect to describe the overall impact of an architectural decision, allowing them to think about whether the negative impacts outweigh the benefits.

This section is also a good place to document the trade-off analysis performed during the decision making process. For example, let say you decide to use asynchronous (Fire-and-Forget) messaging for posting reviews on a website. The justification for this decision is to improve responsiveness (from 3,000 milliseconds down to 25 milliseconds) because users wouldn't have to wait for the actual review to be posted—only for the message to be sent to a queue. One member of your development team argues that this is a bad idea due to the complexity of the error handling associated with an asynchronous request. "What happens if someone posts a review with some bad words?" What this team member doesn't know is that you discussed that very problem with the business stakeholders and other architects when analyzing the trade-offs of this decision, and decided together that it was better to improve responsiveness and deal with complex error handling rather than increase the wait time and provide feedback as to whether the review post was successful or not. If this decision was documented using an ADR, you could have provided this trade-off analysis in the Consequences section, preventing this sort of disagreement.

Compliance

The Compliance section is not one of the standard sections of an ADR, but it was we highly recommend adding. The Compliance section states how the architectural decision will be measured and governed. Will the compliance check for this decision be manual or can it be automated using a fitness function? If it can be automated, the architect can specify how the fitness function will be written, along with any other changes to the code base needed to measure this architectural decision for compliance.

For example, suppose you make the decision within a traditional n-tiered layered architecture (as illustrated in Figure 4-3) that all shared objects used by business objects in the Business layer must reside in the Shared Services layer to isolate and co-locate shared functionality.



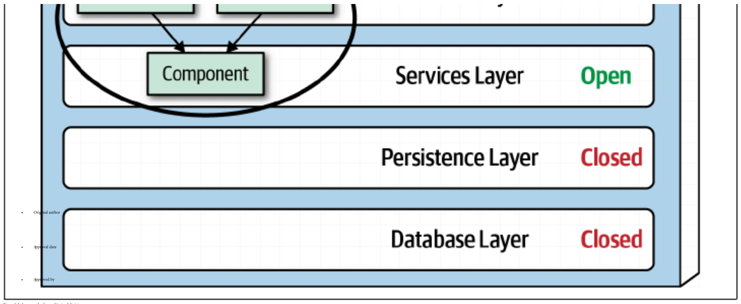


Figure 4-3. An example of an architectural decision

This architectural decision can be measured and governed using a host of automation tools, including ArchUnit in Java and NetArchTest in C#. With ArchUnit in Java, the automated fitness-function test for this architectural decision might look like this:

```
@Test
public void shared_services_should_reside_in_services_layer() {
    classes().that().areAnnotatedWith(SharedService.class)
      .should().resideInAPackage("..services..")
      .check(myClasses);
}
```

This automated fitness function would require writing new stories to create a Java annotation (@SharedService) and add it to all shared classes to support this method of governance.

Notes

Another section that is not part of a standard ADR but that we highly recommend adding is a Notes section. This section includes various metadata about the ADR, such as:

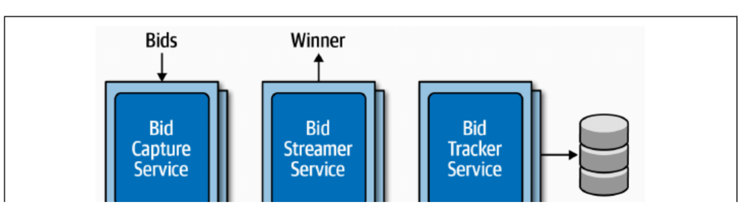
- Original author
- Approval date
- Approved by
- Superseded date
- Last modified date
- Modified by
- Last modification

Even when storing ADRs in a version-control system (such as Git), it's useful to have additional metadata beyond what the repository can support. We recommend adding this section regardless of how and where the architect stores ADRs.

Example

Our example of the ??? auction system includes dozens of architectural decisions. Splitting up the bidder and auctioneer user interfaces, using a hybrid architecture consisting of event-driven and microservices, leveraging the Real-time Transport Protocol (RTP) for video capture, using a single API gateway and using separate queues for messaging are just a few of the architectural decisions an architect would make. Every architectural decision an architect makes, no matter how obvious, should be documented and justified.

Figure 4-4 illustrates one of the architectural decisions within the Going, Going, Gone auction system: using separate point-to-point queues between the bid capture, bid streamer and bid tracker services rather than a single publish-and-subscribe topic (or even REST for that matter).



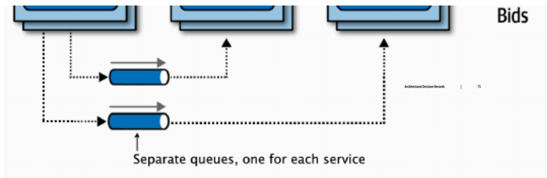


Figure 4-4. Use of pub/sub between services

Without an ADR to justify this decision, others involved with designing and developing this system might disagree and decide to implement it in a different way.

The following is an example of an ADR for this architectural decision:

ADR 76. Separate Queues for Bid Streamer and Bidder Tracker Services

STATUS

Accepted

CONTEXT

The `Bid Capture` service, upon receiving a bid, must forward that bid to the `Bid Streamer` service and the `Bidder Tracker` service. This could be done using a single topic (pub/sub), separate queues (point-to-point) for each service, or REST via the Online Auction API layer.

DECISION

We will use separate queues for the `Bid Streamer` and `Bidder Tracker` services.

The `Bid Capture` service does not need any information from the `Bid Streamer` service or `Bidder Tracker` service (communication is only one-way).

The `Bid Streamer` service must receive bids in the exact order they were accepted by the `Bid Capture` service. Using messaging and queues automatically guarantees the bid order for the stream by leveraging first-in, first out (FIFO) queues.

Multiple bids come in for the same amount (for example, "Do I hear a hundred?"). The `Bid Streamer` service only needs the first bid received for that amount, whereas the `Bidder Tracker` needs all bids received. Using a topic (pub/sub) would require the `Bid Streamer` to ignore bids that are the same as the prior amount, forcing the `Bid Streamer` to store shared state between instances.

The `Bid Streamer` service stores the bids for an item in an in-memory cache, whereas the `Bidder Tracker` stores bids in a database. The `Bidder Tracker` will therefore be slower and might require back pressure. Using a dedicated `Bidder Tracker` queue provides this dedicated back-pressure point.

CONSEQUENCES

We will require clustering and high availability of the message queues.

This decision will require the `Bid Capture` service to send the same information to multiple queues.

Internal bid events will bypass security checks done in the API layer.

UPDATE: Upon review at the January 14, 2026 ARB meeting the ARB decided that this was an acceptable tradeoff and that no additional security checks are needed for bid events between these services.

COMPLIANCE

We will use periodic manual code reviews to ensure that asynchronous pub/sub messaging is being used between the `Bid Capture` service, `Bid Streamer` service, and `Bidder Tracker` service.

NOTES

Author: Subashini Nadella

Approved: ARB Meeting Members, 14 JAN 2026

Last Updated: 14 JAN 2026

Storing ADRs

Once an architect creates an ADR, they need to store it somewhere. Regardless of where that is, each architectural decision should have its own file or wiki page. Some architects like to keep ADRs in the same Git repository as the source code, allowing the team to version and track ADRs as they would source code.

However, for larger organizations, we caution against this practice for several reasons. First, everyone who needs to see the architectural decision might not have access to the Git repository containing the ADRs. Second, the application Git repository is not a good place to store ADRs that have a context outside of it (such as integration architectural decisions, enterprise architectural decisions, or decisions that are common to every application). For these reasons, we recommend storing ADRs in a dedicated ADR Git repository to which everyone has access, a wiki (using a wiki template), or a shared directory on a shared file server that can be accessed easily by a wiki or other document-rendering software.

Figure 4-5 shows what this directory structure (or wiki page navigation structure) might look like.

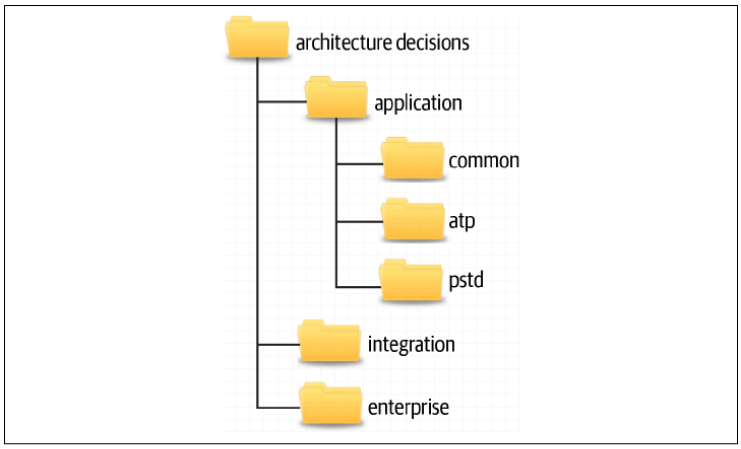


Figure 4-3. Example directory structure for storing ADRs

The application directory contains architectural decisions that are specific to some sort of application (or product) context. This directory is subdivided into further directories:

Common

The *common* subdirectory is for architectural decisions that apply to all applications, such as .NET framework-related classes will contain an annotation (attribute in Java) or attribute (Framework) in C# identifying the class as belonging to the underlying framework stack.

Application

Subdirectories under the application directory correspond to the specific application or system context and contain architectural decisions specific to that application or system (in this example, the ATP and PSTD applications).

Integration

The integration directory contains ADRs that involve communication between application, systems, or services.

Enterprise

Enterprise architecture ADRs are contained within the enterprise directory indicating that these are global architectural decisions impacting all systems and applications. An example of an enterprise architecture ADR would be:ER access to a system database will only be from the owning system,preventing databases from being shared across multiple systems.

When storing ADRs in a wiki, the same structure applies, with each directory structure representing a navigational landing page. Each ADR is represented as a single wiki page within each navigational landing page (Applications, Integration, or Enterprise).

The directory and landing-page names indicated in this section are only recommendations and examples. Choose whatever names fit the company situation, as long as those names are consistent across teams.

ADRs as Documentation

Documenting software architecture has always been difficult. While some standards

are emerging for diagramming architecture (such as software architect Simon Brown's C4 Model or The Open Group ArchiMate standard), no agreed-upon standard exists for documenting software architecture. That's where ADRs come in.

ADRs can be an effective means to document a software architecture. The Context section provides an excellent opportunity to describe the specific area of the system that requires an architectural decision to be made, as well as to describe the alternatives. More importantly, the Decision section describes the reasons why a particular decision is being made, which is by far the best form of architectural documentation. The Consequences section adds the final piece of the puzzle by describing the trade-off analysis for the decision—for example, the reasons (and trade-offs) for choosing performance over scalability.

Using ADRs with Existing Systems

Many architects question the usefulness of ADRs for existing systems. After all, architectural decisions have already been made and the system is in production. Do ADRs serve any real purpose at this point? In fact, they do. Remember, ADRs are more than just documentation—they help architects and developers understand *why* a decision was made and if it was the most appropriate one.

Start by writing some ADRs for the more significant architectural decisions made and questioning whether those decisions are the right ones or not. For example, maybe a group of services are sharing a single database. Why? Is there a good reason? Should the data be broken apart?

Part of the journey of incorporating ADRs into an existing system is doing a little investigative work to uncover these *why* questions. Unfortunately, the person who made the original decision might have left the company a long time ago, so no one knows the answer. In these cases it's up to the architect to identify and analyze the alternatives and the trade-offs of each option and attempt to validate (or invalidate) the existing decision. In either case, writing ADRs for these sorts of significant decisions starts to build up the justifications and rationales (and brain trust) for the system and can help identify architectural inefficiencies and incorrect system design.

Using ADRs for Standards

Very few developers like standards. Unfortunately, standards are sometimes more about control rather than providing a useful purpose. Using ADRs for standards can change this bad practice. For example, the Context section of an ADR describes the situation that is forcing the organization to adopt the particular standard. The Decision section of an ADR can thus indicate not only what the standard is, but more importantly why it needs to exist.

This is a great way to qualify whether a particular standard should even exist in the first place. If an architect can't justify it, then perhaps it isn't a good standard to set and enforce. Furthermore, the more developers understand *why* a particular standard exists, the more likely they are to follow it (and, correspondingly, not challenge it). The Consequences section of an ADR is another great place to qualify whether a standard is valid: it requires the architect to think about and document the standard's implications and consequences, and whether the particular standard should be implemented or not.

Leveraging Generative AI and LLMs in Architectural Decisions

One of the many intriguing aspects of generative AI is whether architects can use it to help make and validate decisions. Should services use messaging, streaming, or event sourcing when sending downstream data? Should the database remain as a single [illegible]

cessing be deployed as a single service or broken apart into multiple services, one for each payment type?

Most architects already know the answer to these questions—it depends! Coming back to our first law of software architecture, *everything in software architecture is a trade-off*. Decisions like these depend on a lot of factors, including the specific context in which the decision is applied. Every situation and every environment is different, which is why there are no "best practices" for these sorts of structural questions.

Most LLMs base their results largely on probability. In other words, what is the most probable answer given the context of the prompt, and what is the "best practice" for this problem? However, probability and "best practices" have no place in making architectural decisions. Answering architectural questions requires a careful analysis of the trade-offs involved and applying a specific business and technical context to identify the most appropriate choice. For example, if the business is most concerned about time-to-market (getting changes and new features out to customers as fast as possible), then *maintainability* is going to be far more important than *performance*,

and will drive most of the decision-making process towards optimizing maintainability.

Architectural decisions require translating business concerns (such as time-to-market or sustained growth) into architectural characteristics (such as maintainability, testability, deployability, and so on). This translation is not always obvious, and getting it right requires years of experience. Once completed, it serves as a basis for trade-off analysis. For example, deciding whether to have a single service for payment process-

ing or a service per payment type boils down to a trade-off between maintainability and performance: a single service provides better performance, but multiple services provides better maintainability.If the business is primarily concerned about time-to-market, maintainability than performance, so separate services would be the appropriate choice for this specific context.

Because of the very specific and individualistic nature of analyzing trade-offs and business context, it is difficult for generative AI as it currently exists to arrive at the most appropriate architectural decision. The best-case scenario, based on recent experiments your authors have done, is to have a generative AI model outline the possible trade-offs of a decision, to assist in identifying any missed trade-offs. While generative AI tools have plenty of knowledge, they lack the intuitive required to make the most appropriate architectural decision.

About the Authors

Mark Richards is an experienced hands-on software architect involved in the archi-

tecture, design, and implementation of microservices and other distributed architec-

tures. He is the founder of *DeveloperToArchitect.com*, a website devoted to assisting

developers in the journey from developer to a software architect.

Neal Ford is director, software architect, and meme wrangler at ThoughtWorks, a

global IT consultancy with an end-to-end software development and delivery philos-

ophy. Before joining ThoughtWorks, Neal was the chief technology officer at

The DVD Group Ltd., a nationally recognized training and development firm.